

13

Dynamics 365 Business Central APIs

In the previous chapter, we saw tips that you can use during development to improve the performance of your AL code.

When working in a Dynamics 365 Business Central project, integration with external systems is often a requirement. Integrations between Dynamics 365 Business Central and other systems and services can be created in different ways. In this chapter, we'll see how to integrate Dynamics 365 Business Central with external applications by using the RESTful APIs exposed by the platform (APIs that conform to the standard REST guidelines), and the focus will be on the following topics:

- Using the OData protocol for APIs
- Handling OAuth 2.0 authentication with Dynamics 365 Business Central APIs
- Using the Dynamics 365 Business Central standard APIs
- Creating custom APIs with Dynamics 365 Business Central for new and existing entities
- Creating applications that use Dynamics 365 Business Central APIs
- Using OData bound and unbound actions
- Using batches for sending API requests in bulk
- Using Dynamics 365 Business Central webhooks
- Using Automation APIs in Dynamics 365 Business Central

By the end of this chapter, you will be able to create RESTful APIs for Dynamics 365 Business Central and use them efficiently for integrations with external applications like third-party systems or services, other APIs, and so on.

Using the OData protocol for APIs

Every client that can make HTTP calls can consume RESTful APIs. By using the GET, POST, PATCH, and DELETE verbs of the HTTP protocol, you can **Create, Read, Update, and Delete (CRUD)** entities. To make integrations with Dynamics 365 Business Central, the OData protocol and RESTful APIs are the recommended tools to work with.

The **Open Data (OData)** protocol is a web protocol that permits you to perform CRUD operations on tabular data with HTTP calls by using **URIs** (short for **Uniform Resource Identifiers**) for resource identification. These URIs, like URLs, serve as OData **endpoints** that identify resources on the web.

At the moment, there is a simple way to expose an object (like a page, a codeunit, or a query object) as an OData endpoint in Dynamics 365 Business Central. First, you open the **Web Services** page, and then you insert a new record by setting the **Object Type** and **Object ID** fields, give it a **Service Name**, and set the **Publish** field to true.

As you can see in the following screenshot, Dynamics 365 Business Central automatically assigns an OData V4 URL to the published entity, and then you can use this published entity as an OData endpoint by performing HTTP REST calls to the provided endpoints:

Dynamics 365 Business Central

Web Services

× If you want to set up an OData connection, for performance and stability reasons consider using an API page instead. Don't show again | API documentation

Search + New Edit List Delete Reload Download Metadata Document More options

Object Type ↑	Object ID	Object Name	Service Name ↑	All Tena...	Publi...	OData V4 URL
Page	5493	cashFlowStatement	ExcelTemplateCashFlowStatement	☐	☒	https://api.businesscentral.dynami...
Page	5503	incomeStatement	ExcelTemplateIncomeStatement	☐	☒	https://api.businesscentral.dynami...
Page	5497	retainedEarningsStatement	ExcelTemplateRetainedEarnings	☐	☒	https://api.businesscentral.dynami...
Page	5502	trialBalance	ExcelTemplateTrialBalance	☐	☒	https://api.businesscentral.dynami...
Page	1320	ExcelTemplateCompanyInfo	ExcelTemplateViewCompanyInform...	☐	☒	https://api.businesscentral.dynami...
Page	2200	Sales Invoice Document API	InvoiceDocument	☒	☒	https://api.businesscentral.dynami...
Page	2201	Sales Invoice Reminder API	InvoiceReminder	☒	☒	https://api.businesscentral.dynami...
→ Page	89	Jobs	Job List	☐	☒	https://api.businesscentral.dynami...
Page	1007	Job Planning Lines	Job Planning Lines	☐	☒	https://api.businesscentral.dynami...
Page	1002	Job Task Lines	Job Task Lines	☐	☒	https://api.businesscentral.dynami...
Page	16	Chart of Accounts	Piano dei conti	☐	☒	https://api.businesscentral.dynami...

Figure 13.1: OData V4 URL for an entity

When calling an OData endpoint, you can apply filters, use grouping, use FlowFilters, and call business logic by using bound actions (we'll discuss them later in this chapter, in the *Using bound actions* section).

While the Web Services page is currently a simple option for us to get started with using the OData protocol, it is important to know that the Web Services page will be deprecated in the future. Once you're comfortable with understanding how to use HTTP REST calls to OData endpoints, you should move on to learning how to use **dedicated API pages**.

These API pages, although slightly more complex, use the same OData protocol for APIs. Learning how to use them will help you avoid deprecation issues, and also allow you to enjoy some performance and stability advantages, which we will now explore.

APIs in Dynamics 365 Business Central use the same OData protocol under the hood, but they have three main advantages when we talk about integration:

- They have **versioning**, which doesn't exist for web services. This is one of the most important things when doing service integration because you need a stable contract. If you have made improvements to an API, you should keep in mind that changing APIs is breaking and should always be done changing the version.
- They are **webhook supported** (you can publish your API page and then call `/api/microsoft/runtime/beta/webhookSupportedResources?$filter=resource eq 'v2.0*'` to verify that the entity is supported by webhooks).
- They have **namespaces** so you can isolate and group your APIs according to their scope or functional area: `{{shortUrl}}/api/APIPublisher/APIGroup/v1.0/customers('01121212')`.



For more information on how to use RESTful APIs in general, I recommend the following link: <https://www.odata.org/getting-started/basic-tutorial/>.

With these advantages in mind, we should now understand that API pages are the most performant solution when creating OData endpoints. The rest of this chapter will proceed using API pages.

Configuring OAuth authentication for Dynamics 365 Business Central APIs

To start using Dynamics 365 Business Central APIs, you first need to be authenticated. Dynamics 365 Business Central SaaS requires that you use the **OAuth** protocol for authentication.

OAuth is an open standard authorization protocol for securing access to third-party services. OAuth never shares passwords between the actors of a process but instead, it sends authorization tokens, and this token is used to prove the authenticity of the identity.

OAuth permits users to sign in to Business Central APIs using their Microsoft 365 or Microsoft Entra ID credentials in an interactive or non-interactive way. Please note that in Dynamics 365 Business Central on-premises, you can still use basic authentication (username and web service access key).

In this section, we'll see how to configure Microsoft Entra ID (previously named Azure Active Directory or Azure AD) for using **Service-to-Service (S2S)** authentication with Dynamics 365 Business Central APIs. S2S authentication is often the best choice for scenarios where integrations need to run without any user interaction. This will be almost every scenario you face when creating integrations in Business Central.

S2S authentication uses the Client Credentials OAuth 2.0 Flow, which enables you to access resources by using the identity of an application. More information about this authentication flow can be found at the following link: <https://learn.microsoft.com/en-us/azure/active-directory/develop/v2-oauth2-client-creds-grant-flow>.

To configure Microsoft Entra and Dynamics 365 Business Central APIs to use OAuth 2.0 with S2S authentication, we need to:

1. Register an application in Microsoft Entra ID.
2. Set the application permissions.
3. Create a client secret.
4. Register a Microsoft Entra ID application in Dynamics 365 Business Central.

Registering an application in Microsoft Entra ID

From the Azure portal, open **Microsoft Entra ID** and select **App registrations** | **New registration**:

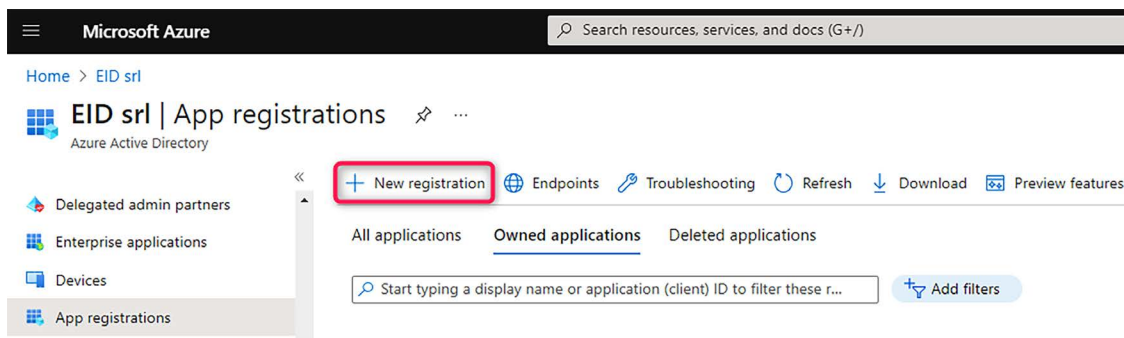


Figure 13.2: App registrations page

Select a name for the app registration and then select **Accounts in this organizational directory only (single tenant)** or **Accounts in any organizational directory (Any Microsoft Entra directory - Multitenant)**, according to your needs. Normally, when you select **Accounts in this organizational directory only (single tenant)**, you allow only users and guests from the tenant where the developer has registered their app. This option is the most common for Line of Business (LOB) applications. When you select **Accounts in any organizational directory (Any Microsoft Entra directory - Multitenant)**, you allow any user from any Microsoft Entra directory to sign in to your multi-tenant application.

Set the **Redirect URI** field with the following value (case-sensitive): <https://businesscentral.dynamics.com/OAuthLanding.htm>.

Then, click on **Register**:

Home > | App registrations >

Register an application

*** Name**

The user-facing display name for this application (this can be changed later).

PACKTD365BCAPI ✓

Supported account types

Who can use this application or access this API?

☐ Accounts in this organizational directory only (EID srl only - Single tenant)

☒ Accounts in any organizational directory (Any Azure AD directory - Multitenant)

☐ Accounts in any organizational directory (Any Azure AD directory - Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

☐ Personal Microsoft accounts only

[Help me choose...](#)

Redirect URI (optional)

We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

Web ✓

Register an app you're working on here. Integrate gallery apps and other apps from outside your organization by adding from [Enterprise applications](#).

By proceeding, you agree to the [Microsoft Platform Policies](#)

Register

Figure 13.3: Configuring the app name and supported account type

From the **Overview** pane that appears after the app registration, copy the value of the **Application (client) ID** field (it will be used at a later time):

Home > | App registrations >

PACKTD365BCAPI

Search << Delete Endpoints Preview features

Overview

Quickstart

Integration assistant

Manage

Branding & properties

Authentication

Essentials

Display name : PACKTD365BCAPI

Application (client) ID : 84a0435b-3ee3-450e-9768-cdaab02cb9a3

Object ID : 181f9e05-4dd3-4ae9-9840-0902c7736ec9

Directory (tenant) ID :

Supported account types : Multiple organizations

Client credentials : Add a certificate or secret

Redirect URIs : 1 web, 0 spa, 0 public client

Application ID URI : Add an Application ID URI

Managed application in L... : PACKTD365BCAPI

Figure 13.4: Finding the application ID

The app registration is now complete. In the next section, we'll see how to set up the permissions.

Setting the application permissions

To set the permissions for the newly created application registration, select **API permissions** and then click on **Add a permission**:

Home > App registrations > PACKTD365BCAPI

PACKTD365BCAPI | API permissions

Search « Refresh Got feedback?

Overview

Quickstart

Integration assistant

Manage

Branding & properties

Authentication

Certificates & secrets

Token configuration

API permissions

Expose an API

Configured permissions

Applications are authorized to call APIs when they are granted permissions by users/admins as part of the consent process. The list of configured permissions should include all the permissions the application needs. [Learn more about permissions and consent](#)

+ Add a permission ✓ Grant admin consent for

API / Permissions name	Type	Description	Admin consent requ...	Status
Microsoft Graph (1)				...
User.Read	Delegated	Sign in and read user profile	No	...

To view and manage consented permissions for individual apps, as well as your tenant's consent settings, try [Enterprise applications](#).

Figure 13.5: Configuring API permissions

From the **Microsoft APIs** list, select **Dynamics 365 Business Central** and then select **Application permissions**. Then assign the **API.ReadWrite.All** permission and select **Add permissions**:

Request API permissions

[< All APIs](#)



Dynamics 365 Business Central

<https://dynamics.microsoft.com/business-central/overview/> [Docs](#)

What type of permissions does your application require?

Delegated permissions

Your application needs to access the API as the signed-in user.

Application permissions

Your application runs as a background service or daemon without a signed-in user.

Select permissions

expand all

Start typing a permission to filter these results

Permission	Admin consent required
Other permissions	
☐ app_access ⓘ Access according to the application's permissions in Dynamics 365 Business Central	Yes
AdminCenter	
☐ AdminCenter.ReadWrite.All ⓘ Full access to Admin Center API	Yes
API (1)	
☒ API.ReadWrite.All ⓘ Full access to web services API	Yes

Add permissions

Discard

Figure 13.6: Adding Business Central API permissions

A new permission for Dynamics 365 Business Central will be added. The **Status** field will say **Not granted for the current organization**. The grant will be assigned in Dynamics 365 Business Central:

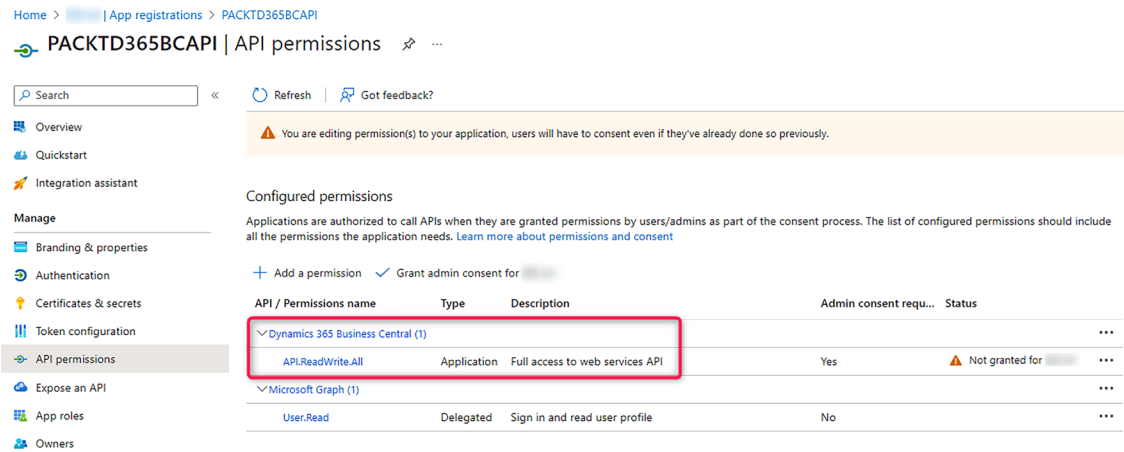


Figure 13.7: Viewing the Business Central API permission

Now we have configured the permissions for our app registration. In the next section, we'll see how to create the secret key.

Creating a client secret

To create a client secret for your registered application, select **Certificates & secrets** and click on **New client secret**. Then set the secret name and the corresponding due date (maximum 24 months) and click on **Add**:

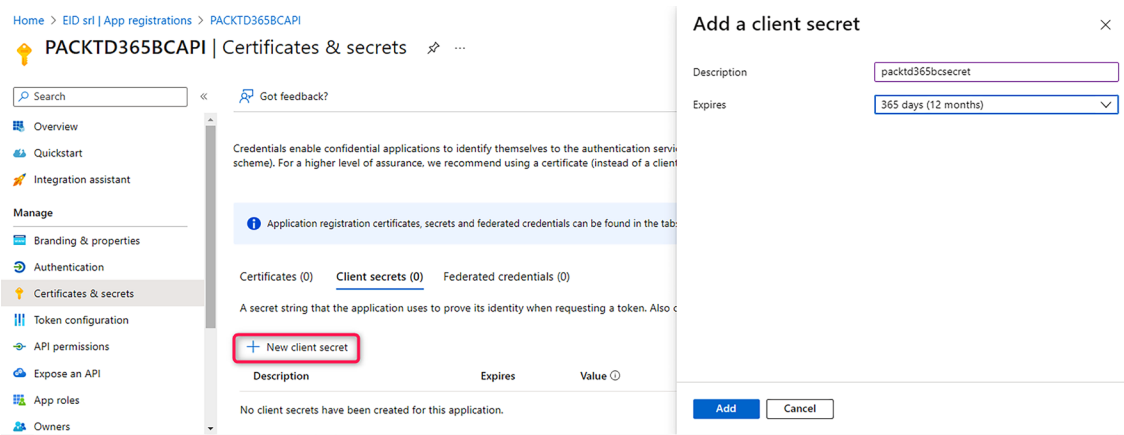


Figure 13.8: Button for adding a client secret

A new secret is generated. Immediately copy the generated secret value (the **Value** field). Its value will never be shown again.



The secret must be renewed when the due date expires. It cannot be set as unlimited expiration.

Certificates (0) Client secrets (1) Federated credentials (0)

A secret string that the application uses to prove its identity when requesting a token. Also can be referred to as application password.

+ New client secret

Description	Expires	Value ⓘ	Secret ID
packtd365bcsecret	8/2/2024	Zmz8Q~eenxvBTMjka9DniLI	55c6649f-84fb-449f-8ab0-eb86c

Figure 13.9: Viewing a client secret value

Now we have everything configured. In the next section, we’ll see how to register the application in Dynamics 365 Business Central.

Microsoft Entra application registration in Dynamics 365 Business Central

Now you’re ready to register your Microsoft Entra application in Dynamics 365 Business Central. To do that, open Dynamics 365 Business Central and search for **Microsoft Entra Applications**. On the **Microsoft Entra Application Card** page, click on **New** and **Create a new record** as follows:

- **Client ID:** Copy the value of the application (client) ID generated from the Azure AD app registration.
- **Description:** Create a description
- **State:** Enabled

In the **User Permission Sets** section, set the API permissions as needed for your scenario. Then select **Grant Consent**.

←

Microsoft Entra Application Card

✓ Saved

PACKT D365BC API

⚙️ Grant Consent

More options

General

Client ID

{84a0435b-3ee3-450e-9768-cd...

Description

PACKT D365BC API

State

Enabled

Contact Information

Extension

App ID

{00000000-0000-0000-0000-00...

App Name

User information

User ID

{bd44f224-157f-4df1-9fa7-b646...

User Name

PACKT D365BC API

You must set the State field to Disabled before you can mak...

User Groups

Manage

Code ↑

Name

Company Name ↑

→

CRONUS IT

Figure 13.10: Granting API permissions

A new login request will appear in order to authorize the app registration. The user that needs to perform the authorization must be a Global Administrator, Application Administrator, or Cloud Application Administrator:

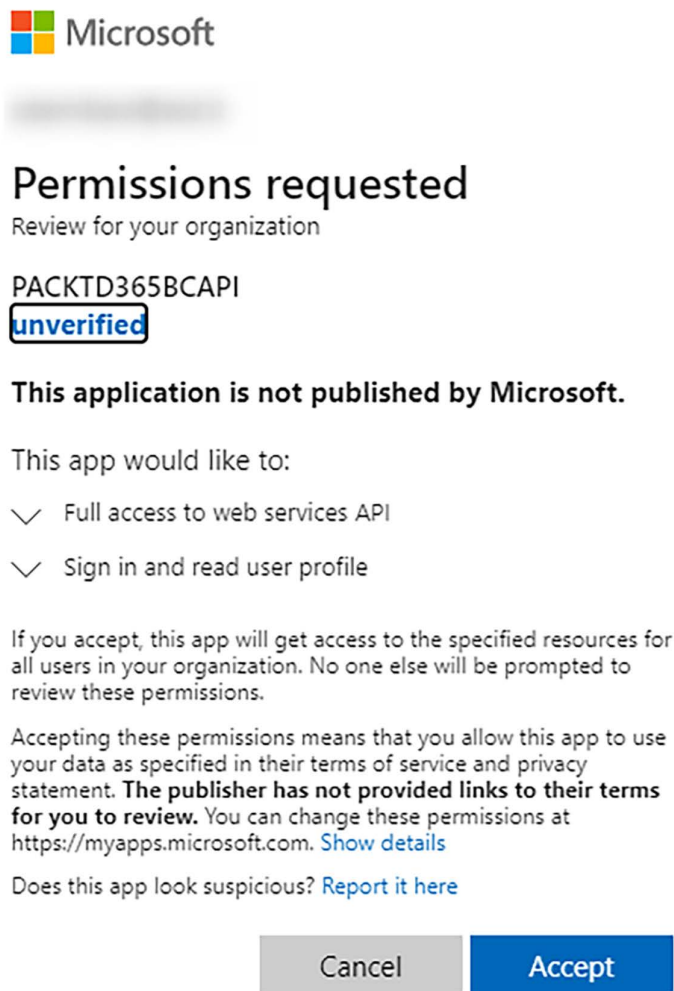


Figure 13.11: Authorizing app registration

Click on **Accept**; if the registration is successfully executed, you will have the following confirmation message:

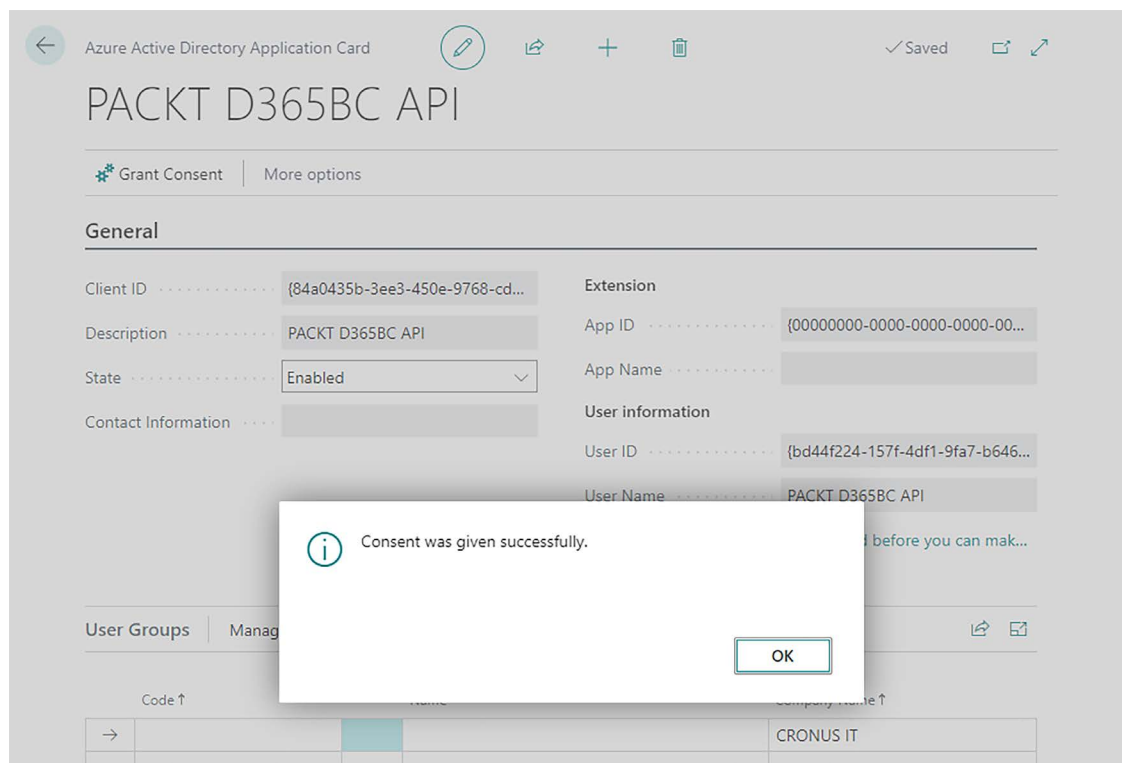


Figure 13.12: Granting app registration

Your Microsoft Entra application is now registered in Dynamics 365 Business Central and you can start using it for making API calls. We'll see that in the next sections.

Acquiring an authentication token from Microsoft Entra ID

In order to use Dynamics 365 Business Central APIs, you need to request an authentication token to Microsoft Entra ID. An authentication result from Microsoft Entra contains an access token and an ID token. The **access token** is the token we need to use to call Dynamics 365 Business Central APIs (as a Bearer token in the Authorization header of the incoming API call):

```
Authorization: Bearer <token>
```

Remember that an access token is short-lived (one hour by default and one day maximum). When it expires, the client application needs to request a new access token.

Here is an example of HTTP requests that you need to send in order to obtain an authentication token with S2S authentication flow:



I'm using the Rest Client extension from Visual Studio Code to send the needed HTTP requests for the authorization flow. You can obtain the extension from the Visual Studio Code Marketplace or from here: <https://marketplace.visualstudio.com/items?itemName=humao.rest-client>.

```
@tenantId = <tenant id>
@clientId = <client id>
@clientSecret = <client secret>
@baseUri = https://api.businesscentral.dynamics.com
@scope = {{baseUri}}/.default
@bcEnvironmentName = production
@url = {{baseUri}}/v2.0/{{bcEnvironmentName}}/api/v2.0

### Define the Business Central entity name to work with
@EntityName =customers

### Authentication
POST https://login.microsoftonline.com/{{tenantId}}/oauth2/v2.0/token HTTP/1.1
Content-type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_id={{clientId}}
&client_secret={{clientSecret}}
&scope={{scope}}

### Variable Response
@accessHeader = Bearer {{auth.response.body$.access_token}}

### Retrieving Business Central companies
GET {{url}}/companies HTTP/1.1
Authorization: {{accessHeader}}

### Retrieving the company id
@companyId = {{GetCompanies.response.body.value.[0].id}}
@companyUrl = {{url}}/companies/{{companyId}}

### Get entities
GET {{companyUrl}}/{{entityName}} HTTP/1.1
Authorization: {{accessHeader}}
```

For scenarios where you need to use OAuth authentication from AL code (for example, a custom extension that needs to call an external API secured by Microsoft Entra ID authentication), you can use the **OAuth 2.0 module** of the **System** application in the following way:

```
var
    OAuth2: Codeunit OAuth2;
    ClientId: Text;
    ClientSecret: Text;
    MicrosoftOAuth2Url: Text;
    RedirectURL: Text;
    ResourceURL: Label 'https://graph.microsoft.com/';
    AccessToken: Text;

local procedure ClientCredentials()
var
    Scopes: List of [Text];
begin
    Scopes.Add(ResourceURL + '.default');

    OAuth2.AcquireTokenWithClientCredentials(ClientId, ClientSecret,
MicrosoftOAuth2Url, RedirectURL, Scopes, AccessToken);

end;
```

In this section, you learned how to acquire a security token from Microsoft Entra ID via HTTP calls or via AL Language in your custom extensions. The acquired security token must then be used to authenticate your API calls. In the next section, we'll see how we can use standard Dynamics 365 Business Central APIs.

Using Dynamics 365 Business Central standard APIs

The Dynamics 365 Business Central platform exposes some standard entities (like Customer and Vendor) as RESTful APIs. The exposed standard entities are listed and documented at the following URL: <https://learn.microsoft.com/en-us/dynamics365/business-central/dev-itpro/api-reference/v2.0/>.

Dynamics 365 Business Central API endpoints have the following format:

Endpoint URL section	Description
https://api.businesscentral.dynamics.com	Dynamics 365 Business Central base URL (the same for standard and custom APIs)
/v2.0	API version
/your tenant domain	Domain name or ID of the Dynamics 365 Business Central tenant
/environment name	Name of the environment (production, sandbox, and so on). This can be retrieved from the Dynamics 365 Business Central Admin portal
/api	Fixed value
/v2.0	Indicates the version of the API in use

Table 13.1: Endpoint formats

At the time of writing this book, Dynamics 365 Business Central APIs are on endpoint version 2.0, and `API version_number = 2.0`.

You can see the metadata of the exposed OData entities at the following URL:

```
GET https://api.businesscentral.dynamics.com/v2.0/{tenant Id}/{environment name}/api/v2.0/$metadata
```

This call returns an EDMX document that contains a complete description of the feeds, types, properties, and relationships exposed by the service in the **entity data model (EDM)**.

Note that if you’re using OAuth authentication, the tenant ID can also be omitted:

```
GET https://api.businesscentral.dynamics.com/v2.0/{environment name}/api/v1.0/$metadata
```

To start calling Dynamics 365 Business Central APIs, you need to first acquire a security token via OAuth, as explained in the previous section.

When you have the security token, the next thing that you need to work with a Dynamics 365 Business Central entity exposed as an API is a specific company ID (data in Dynamics 365 Business Central is per company). To retrieve the list of available companies on your Dynamics 365 Business Central tenant, you need to send an HTTP GET request to the `/companies` API endpoint. An example of this API call is as follows:

```
GET https://api.businesscentral.dynamics.com/v2.0/<tenantID>/production/api/v2.0/companies
Authorization: Bearer {{token}}
```

This is the response we receive:

```

1  HTTP/1.1 200 OK
2  Transfer-Encoding: chunked
3  Content-Type: application/json; odata.metadata=minimal; odata.streaming=true
4  Content-Encoding: gzip
5  Vary: Accept-Encoding
6  Server: Microsoft-HTTPAPI/2.0
7  Access-Control-Allow-Headers: Origin, X-Requested-With, Authorization
8  Access-Control-Allow-Origin: *
9  Access-Control-Allow-Credentials: true
10 ms-correlation-x: 06e7555b-1ecd-cf84-3db1-f7e287ec4087
11 mise-correlation-id: 688fe97b-5e2f-4e09-9011-9c55ae2a0ff8
12 OData-Version: 4.0
13 Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version, ms-correlation-x
14 request-id: 81318b12-97e9-4397-9e67-375a3b830a57
15 x-content-type-options: nosniff
16 Strict-Transport-Security: max-age=31536000; includeSubDomains
17 Date: Mon, 17 Jul 2023 15:45:50 GMT
18 Connection: close
19
20 {
21   "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/US$andbox/api/v2.0/$metadata#companies",
22   "value": [
23     {
24       "id": "18bb64bd-e06c-ed11-81b5-6045bd8e591f",
25       "systemVersion": "22.2.56969.57282",
26       "timestamp": 3807,
27       "name": "CRONUS USA, Inc.",
28       "displayName": "",
29       "businessProfileId": "",
30       "systemCreatedAt": "2022-11-25T16:46:45.96Z",
31       "systemCreatedBy": "00000000-0000-0000-0000-000000000001",
32       "systemModifiedAt": "2022-11-25T16:46:45.96Z",
33       "systemModifiedBy": "00000000-0000-0000-0000-000000000001"
34     },
35     {
36       "id": "989d7ecd-e06c-ed11-81b5-6045bd8e591f",
37       "systemVersion": "22.2.56969.57282",
38       "timestamp": 3808,
39       "name": "My Company",
40       "displayName": "",
41       "businessProfileId": "",
42       "systemCreatedAt": "2022-11-25T16:47:11.433Z",
43       "systemCreatedBy": "00000000-0000-0000-0000-000000000001",
44       "systemModifiedAt": "2022-11-25T16:47:11.433Z",
45       "systemModifiedBy": "00000000-0000-0000-0000-000000000001"
46     }
47   ]
48 }

```

Figure 13.13: Retrieving company IDs with a GET request

If we want to retrieve the list of Customer records for a specific company (for example, Cronus USA, Inc.), we need to send an HTTP GET request to the following API endpoint:

```
GET https://api.businesscentral.dynamics.com/v2.0/<tenantID>/production/api/v2.0/companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/customers
```



```
Authorization: Bearer {{token}}
```

This is the response we receive from it:

```

4 Content-Encoding: gzip
5 Vary: Accept-Encoding
6 Server: Microsoft-HTTPAPI/2.0
7 Access-Control-Allow-Headers: Origin, X-Requested-With, Authorization
8 Access-Control-Allow-Origin: *
9 Access-Control-Allow-Credentials: true
10 ms-correlation-x: 68264f46-6421-16ed-2dcf-62fc29eba995
11 mise-correlation-id: e3e5c2ee-0823-4411-a657-d3719d4eaa61
12 OData-Version: 4.0
13 Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version, ms-correlation-x
14 request-id: eb5afa59-7689-4cc1-8639-7dbf57ee64f5
15 x-content-type-options: nosniff
16 Strict-Transport-Security: max-age=31536000; includeSubDomains
17 Date: Mon, 17 Jul 2023 15:49:25 GMT
18 Connection: close
19
20 {
21   "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/USSandbox/api/v2.0/$metadata#companies(18b
b64bd-e06c-ed11-81b5-6045bd8e591f)/customers",
22   "value": [
23     {
24       "@odata.etag": "W/\"JzE5OzIwNjM2NDIxODIxNzAwNTI5NjQxOzAwOyc=\"",
25       "id": "71b117ef-e06c-ed11-81b5-6045bd8e591f",
26       "number": "10000",
27       "displayName": "Adatum Corporation",
28       "type": "Company",
29       "addressLine1": "192 Market Square",
30       "addressLine2": "",
31       "city": "Atlanta",
32       "state": "GA",
33       "country": "US",
34       "postalCode": "31772",
35       "phoneNumber": "",
36       "email": "robert.townes@contoso.com",
37       "website": "",
38       "salespersonCode": "JO",
39       "balanceDue": 0,
40       "creditLimit": 0,
41       "taxliable": true,
42       "taxAreaId": "ae7f10f5-e06c-ed11-81b5-6045bd8e591f",
43       "taxAreaDisplayName": "ATLANTA, GA",
44       "taxRegistrationNumber": "",
45       "currencyId": "00000000-0000-0000-0000-000000000000",
46       "currencyCode": "USD",
47       "paymentTermsId": "80b017ef-e06c-ed11-81b5-6045bd8e591f",
48       "shipmentMethodId": "00000000-0000-0000-0000-000000000000",
49       "paymentMethodId": "00000000-0000-0000-0000-000000000000",
50       "blocked": "_x0020_",
51       "lastModifiedDate": "2022-11-25T16:49:20.203Z"
52     },

```

Figure 13.14: Retrieving customer records with a GET request

As you can see, the output (value field) is a JSON array of Customer records.

You can also apply filters when calling the APIs. For example, here, we retrieve all Item records where unitPrice is greater than 100:

```
GET https://api.businesscentral.dynamics.com/v2.0/<tenantID>/production/api/
v2.0/companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/items?$filter=unitPrice%20
gt%20100

Authorization: Bearer {{token}}
```

This is the response:

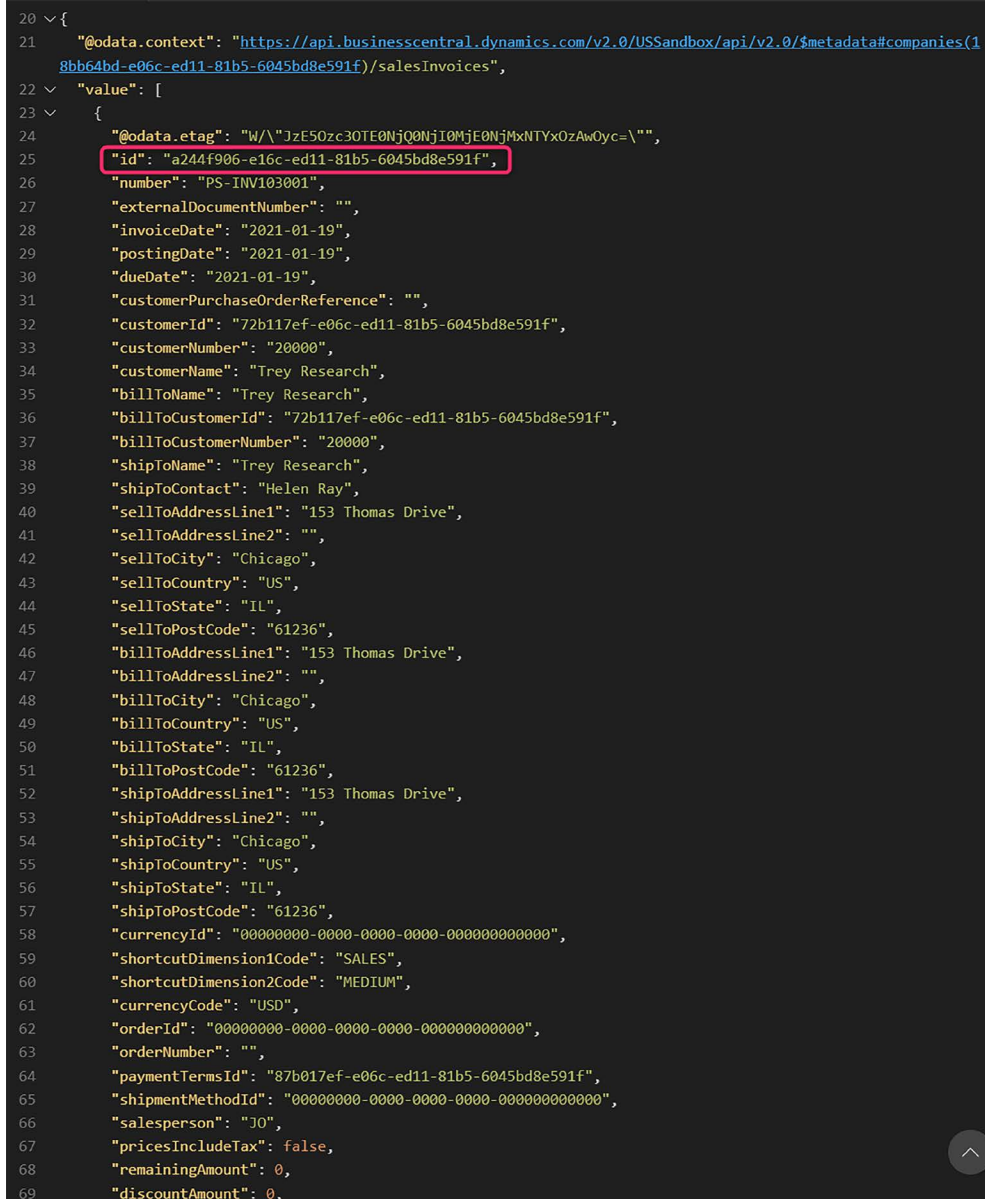
```
1 HTTP/1.1 200 OK
2 Transfer-Encoding: chunked
3 Content-Type: application/json; odata.metadata=minimal; odata.streaming=true
4 Content-Encoding: gzip
5 Vary: Accept-Encoding
6 Server: Microsoft-HTTPAPI/2.0
7 Access-Control-Allow-Headers: Origin, X-Requested-With, Authorization
8 Access-Control-Allow-Origin: *
9 Access-Control-Allow-Credentials: true
10 ms-correlation-x: 3812c1a4-9786-e579-0ba2-91a3a271a29c
11 mise-correlation-id: 934454c4-549b-4bd2-ac82-83b003d62d34
12 OData-Version: 4.0
13 Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version, ms-correlation-x
14 request-id: 529ba83b-71bd-4f56-8f81-7102ac45c555
15 x-content-type-options: nosniff
16 Strict-Transport-Security: max-age=31536000; includeSubDomains
17 Date: Mon, 17 Jul 2023 15:53:38 GMT
18 Connection: close
19
20 {
21   "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/US Sandbox/api/v2.0/$metadata#companies(18b
b64bd-e06c-ed11-81b5-6045bd8e591f)/items",
22   "value": [
23     {
24       "@odata.etag": "W/\"JzIwOzE2OTA3ODI1MDQzMTYyMzYzODEzMTswMDsn\"",
25       "id": "7bb117ef-e06c-ed11-81b5-6045bd8e591f",
26       "number": "1896-S",
27       "displayName": "ATHENS Desk",
28       "type": "Inventory",
29       "itemCategoryId": "478110f5-e06c-ed11-81b5-6045bd8e591f",
30       "itemCategoryCode": "TABLE",
31       "blocked": false,
32       "gtin": "",
33       "inventory": 4,
34       "unitPrice": 1000.8,
35       "priceIncludesTax": false,
36       "unitCost": 780.7,
37       "taxGroupId": "be7f10f5-e06c-ed11-81b5-6045bd8e591f",
38       "taxGroupCode": "FURNITURE",
39       "baseUnitOfMeasureId": "c77d10f5-e06c-ed11-81b5-6045bd8e591f",
40       "baseUnitOfMeasureCode": "PCS",
41       "generalProductPostingGroupId": "3b7f10f5-e06c-ed11-81b5-6045bd8e591f",
42       "generalProductPostingGroupCode": "RETAIL",
43       "inventoryPostingGroupId": "b67d10f5-e06c-ed11-81b5-6045bd8e591f",
44       "inventoryPostingGroupCode": "RESALE",
45       "lastModifiedDateTime": "2022-11-25T16:52:34.62Z"
46     }
47   ]
48 }
```

Figure 13.15: Retrieving item records with a GET request

To have the list of sales invoices, you can perform a GET request to the following endpoint:

```
GET https://api.businesscentral.dynamics.com/v2.0/<tenantID>/production/api/
v2.0/companies(18bb64bd-e06c-ed11-81b5-604
5bd8e591f)/salesInvoices
Authorization: Bearer {{token}}
```

The following screenshots shows the results of this request:



```
20 {
21   "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/US$andbox/api/v2.0/$metadata#companies(1
8bb64bd-e06c-ed11-81b5-6045bd8e591f)/salesInvoices",
22   "value": [
23     {
24       "@odata.etag": "W/\"JzE50zc30TE0NjQ0NjI0MjE0NjMxNTYxOzAwOyc=\"",
25       "id": "a244f906-e16c-ed11-81b5-6045bd8e591f",
26       "number": "PS-INV103001",
27       "externalDocumentNumber": "",
28       "invoiceDate": "2021-01-19",
29       "postingDate": "2021-01-19",
30       "dueDate": "2021-01-19",
31       "customerPurchaseOrderReference": "",
32       "customerId": "72b117ef-e06c-ed11-81b5-6045bd8e591f",
33       "customerNumber": "20000",
34       "customerName": "Trey Research",
35       "billToName": "Trey Research",
36       "billToCustomerId": "72b117ef-e06c-ed11-81b5-6045bd8e591f",
37       "billToCustomerNumber": "20000",
38       "shipToName": "Trey Research",
39       "shipToContact": "Helen Ray",
40       "sellToAddressLine1": "153 Thomas Drive",
41       "sellToAddressLine2": "",
42       "sellToCity": "Chicago",
43       "sellToCountry": "US",
44       "sellToState": "IL",
45       "sellToPostCode": "61236",
46       "billToAddressLine1": "153 Thomas Drive",
47       "billToAddressLine2": "",
48       "billToCity": "Chicago",
49       "billToCountry": "US",
50       "billToState": "IL",
51       "billToPostCode": "61236",
52       "shipToAddressLine1": "153 Thomas Drive",
53       "shipToAddressLine2": "",
54       "shipToCity": "Chicago",
55       "shipToCountry": "US",
56       "shipToState": "IL",
57       "shipToPostCode": "61236",
58       "currencyId": "00000000-0000-0000-0000-000000000000",
59       "shortcutDimension1Code": "SALES",
60       "shortcutDimension2Code": "MEDIUM",
61       "currencyCode": "USD",
62       "orderId": "00000000-0000-0000-0000-000000000000",
63       "orderNumber": "",
64       "paymentTermsId": "87b017ef-e06c-ed11-81b5-6045bd8e591f",
65       "shipmentMethodId": "00000000-0000-0000-0000-000000000000",
66       "salesperson": "JO",
67       "pricesIncludeTax": false,
68       "remainingAmount": 0,
69       "discountAmount": 0,
```

Figure 13.16: Retrieving sales invoices with a GET request

Dynamics 365 Business Central standard APIs also support features such as **expand**, in which, in a single call, you can expand the relationships between entities, and retrieve the main entity along with the related entities. For example, to retrieve a sales invoice and all of its sales invoice line records in a single HTTP call, you can perform an HTTP GET call to the following API endpoint:

```
GET https://api.businesscentral.dynamics.com/v2.0/<tenantID>/production/api/v2.0/companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/salesInvoices(a244f906-e16c-ed11-81b5-6045bd8e591f)?$expand=salesInvoiceLines
Authorization: Bearer {{token}}
```

As a result, you have a single JSON response object with the sales invoice header and its related sales invoice line detail. Here is the header object:

```
20 {
21   "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/US$andbox/api/v2.0/$metadata#companies(18b
22     b64bd-e06c-ed11-81b5-6045bd8e591f)/salesInvoices/$entity",
23   "@odata.etag": "W/\"JzE5Ozc3OTE0NjI0MjE0NjE0NjE0NjE0NTYxOzAwOyc=\"",
24   "id": "a244f906-e16c-ed11-81b5-6045bd8e591f",
25   "number": "PS-INV103001",
26   "externalDocumentNumber": "",
27   "invoiceDate": "2021-01-19",
28   "postingDate": "2021-01-19",
29   "dueDate": "2021-01-19",
30   "customerPurchaseOrderReference": "",
31   "customerId": "72b117ef-e06c-ed11-81b5-6045bd8e591f",
32   "customerNumber": "20000",
33   "customerName": "Trey Research",
34   "billToName": "Trey Research",
35   "billToCustomerId": "72b117ef-e06c-ed11-81b5-6045bd8e591f",
36   "billToCustomerNumber": "20000",
37   "shipToName": "Trey Research",
38   "shipToContact": "Helen Ray",
39   "sellToAddressLine1": "153 Thomas Drive",
40   "sellToAddressLine2": "",
41   "sellToCity": "Chicago",
42   "sellToCountry": "US",
43   "sellToState": "IL",
44   "sellToPostCode": "61236",
45   "billToAddressLine1": "153 Thomas Drive",
46   "billToAddressLine2": "",
47   "billToCity": "Chicago",
48   "billToCountry": "US",
49   "billToState": "IL",
50   "billToPostCode": "61236",
51   "shipToAddressLine1": "153 Thomas Drive",
52   "shipToAddressLine2": "",
53   "shipToCity": "Chicago",
54   "shipToCountry": "US",
55   "shipToState": "IL",
56   "shipToPostCode": "61236",
57   "currencyId": "00000000-0000-0000-0000-000000000000",
58   "shortcutDimension1Code": "SALES",
59   "shortcutDimension2Code": "MEDIUM",
60   "currencyCode": "USD",
61   "orderId": "00000000-0000-0000-0000-000000000000",
62   "orderNumber": "",
63   "paymentTermsId": "87b017ef-e06c-ed11-81b5-6045bd8e591f",
64   "shipmentMethodId": "00000000-0000-0000-0000-000000000000",
65   "salesperson": "JO",
66   "pricesIncludeTax": false,
67   "remainingAmount": 0,
68   "discountAmount": 0,
69   "discountAppliedBeforeTax": true,
```

Figure 13.17: Retrieving a sales invoice header with a GET request

Also, here are the related line's details:

```

59  "currencyCode": "USD",
60  "orderId": "00000000-0000-0000-0000-000000000000",
61  "orderNumber": "",
62  "paymentTermsId": "87b017ef-e06c-ed11-81b5-6045bd8e591f",
63  "shipmentMethodId": "00000000-0000-0000-0000-000000000000",
64  "salesperson": "JO",
65  "pricesIncludeTax": false,
66  "remainingAmount": 0,
67  "discountAmount": 0,
68  "discountAppliedBeforeTax": true,
69  "totalAmountExcludingTax": 164.7,
70  "totalTaxAmount": 8.24,
71  "totalAmountIncludingTax": 172.94,
72  "status": "Paid",
73  "lastModifiedDateTime": "2022-11-25T16:50:27.647Z",
74  "phoneNumber": "",
75  "email": "helen.ray@contoso.com",
76  "salesInvoiceLines": [
77    {
78      "@odata.etag": "W/\\"JzIwOzE1OTY1Nzc0NTM2MDI5Mzg0NzQ3MTswMDsn\"",
79      "id": "e0a67029-e16c-ed11-81b5-6045bd8e591f",
80      "documentId": "a244f906-e16c-ed11-81b5-6045bd8e591f",
81      "sequence": 10000,
82      "itemId": "81b117ef-e06c-ed11-81b5-6045bd8e591f",
83      "accountId": "00000000-0000-0000-0000-000000000000",
84      "lineType": "Item",
85      "lineObjectNumber": "1928-S",
86      "description": "AMSTERDAM Lamp",
87      "unitOfMeasureId": "c77d10f5-e06c-ed11-81b5-6045bd8e591f",
88      "unitOfMeasureCode": "PCS",
89      "quantity": 3,
90      "unitPrice": 54.9,
91      "discountAmount": 0,
92      "discountPercent": 0,
93      "discountAppliedBeforeTax": false,
94      "amountExcludingTax": 164.7,
95      "taxCode": "FURNITURE",
96      "taxPercent": 5,
97      "totalTaxAmount": 8.24,
98      "amountIncludingTax": 172.94,
99      "invoiceDiscountAllocation": 0,
100     "netAmount": 164.7,
101     "netTaxAmount": 8.24,
102     "netAmountIncludingTax": 172.94,
103     "shipmentDate": "2022-11-25",
104     "itemVariantId": "00000000-0000-0000-0000-000000000000",
105     "locationId": "00000000-0000-0000-0000-000000000000"
106   }
107 ]
108 }

```

Figure 13.18: Retrieving sales invoice line details with a GET request

Dynamics 365 Business Central APIs also support the \$select OData clause, which permits you to retrieve only the specified fields of a record set. For example, we can modify the previous API call to retrieve the sales invoice details by specifying only a small set of fields that we need in the following way:

```
GET https://api.businesscentral.dynamics.com/v2.0/<tenantID>/production/
api/v2.0/companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/salesInvoices
?$select=id,number,invoiceDate,customerId,customerNumber,customerName
Authorization: Bearer {{token}}
```

And the response will be the following JSON:

```
{
  "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/US Sandbox/api/v2.0/$metadata#companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/salesInvoices",
  "value": [
    {
      "@odata.etag": "W/\"JzE50zc30TE0NjQ0NjI0MjE0NjMxNTYxOzAwOyc=\"",
      "id": "a244f906-e16c-ed11-81b5-6045bd8e591f",
      "number": "PS-INV103001",
      "invoiceDate": "2021-01-19",
      "customerId": "72b117ef-e06c-ed11-81b5-6045bd8e591f",
      "customerNumber": "20000",
      "customerName": "Trey Research"
    },
    {
      "@odata.etag": "W/\"JzIwOzE3MTY5MzQ4ODU1MzA1MDczMDA0MTswMDsn\"",
      "id": "a444f906-e16c-ed11-81b5-6045bd8e591f",
      "number": "PS-INV103002",
      "invoiceDate": "2021-01-20",
      "customerId": "71b117ef-e06c-ed11-81b5-6045bd8e591f",
      "customerNumber": "10000",
      "customerName": "Adatum Corporation"
    },
    {
      "@odata.etag": "W/\"JzE50zg1MzIyMDE2ODMzMzk3ODIwMTgxOzAwOyc=\"",
      "id": "a644f906-e16c-ed11-81b5-6045bd8e591f",
      "number": "PS-INV103003",
      "invoiceDate": "2021-01-21",
      "customerId": "73b117ef-e06c-ed11-81b5-6045bd8e591f",
      "customerNumber": "30000",
      "customerName": "School of Fine Art"
    },
    {
      "@odata.etag": "W/\"JzE50zg1MTYxNzk2NDI4MDYyMzQ0MjQxOzAwOyc=\"",
      "id": "a844f906-e16c-ed11-81b5-6045bd8e591f",
      "number": "PS-INV103004",
      "invoiceDate": "2021-01-22",
      "customerId": "75b117ef-e06c-ed11-81b5-6045bd8e591f",
      "customerNumber": "50000",
      "customerName": "Relecloud"
    }
  ]
}
```

Figure 13.19: Retrieving specific lines with GET request

After calling a Dynamics 365 Business Central API, you can then parse the JSON response as per your needs.

In the next section, we will see how to create a custom API page for a new entity added to Dynamics 365 Business Central and how to create a new API page for an existing entity.

Creating a custom API in Dynamics 365 Business Central

With Dynamics 365 Business Central extensions, you can create custom entities and you can also expose those custom entities as RESTful APIs. Custom entities are entities that don't exist in the standard product, which will be useful for you when you need an entity for a particular project or a particular customization.

To create a new API in Dynamics 365 Business Central, you need to define a new Page object with PageType = API. To do this, you can use the tpage snippet and then select **Page of type API**, as follows:

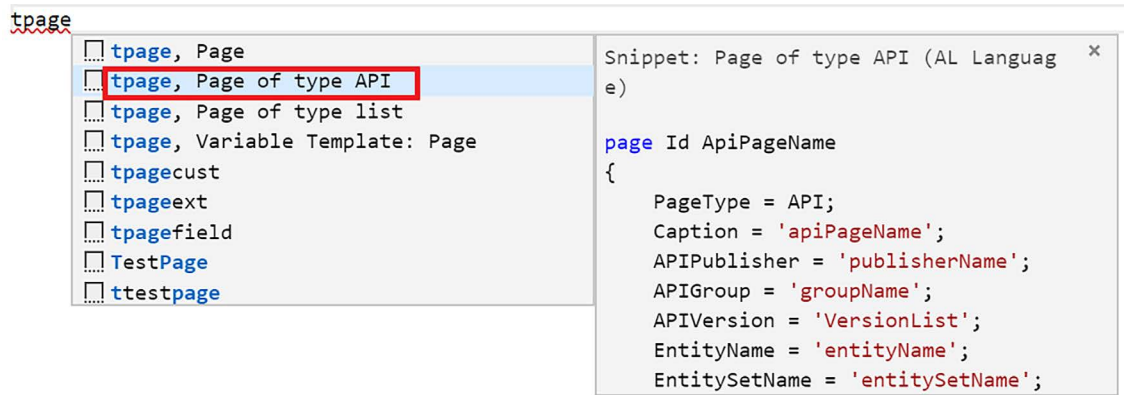


Figure 13.20: Defining a new Page object

When creating API pages, remember the following:

- Fields must have a name in the REST API-compliant format (only alphanumeric values, and no spaces or special characters (camelCase)).
- You should always expose the ID of the entity (SystemId) and this is the recommended OData key to use.
- When you insert, modify, or delete an entity through APIs, triggers on the underlying table are not executed. You need to call the table's trigger by handling the corresponding triggers at the page level.
- If needed, in the OnModify trigger of the API page, you need to handle the possibility of renaming a record (an API call via a record ID can issue a primary key rename).

Here, we'll see two main scenarios:

- How to implement an API for a custom entity (assuming that there's an extension that adds the Car entity to Dynamics 365 Business Central for managing car details inside the ERP)
- How to implement a new API for an existing entity

We will look into each of these scenarios in the following sections.

Implementing a new API for a custom entity

In this example, we will be creating a new entity in Dynamics 365 Business Central to handle the details of cars, and this entity will also be exposed as an API for external applications:

1. To do this, we first create a new Car table, as follows:

```
table 50200 Car
{
    DataClassification = CustomerContent;
    Caption = 'Car';
    LookupPageId = "Car List";
    DrillDownPageId = "Car List";
    fields
    {
        field(1; ModelNo; Code[20])
        {
            Caption = 'Model No.';
            DataClassification = CustomerContent;
        }
        field(2; Description; Text[100])
        {
            Caption = 'Description';
            DataClassification = CustomerContent;
        }
        field(3; Brand; Code[20])
        {
            Caption = 'Brand';
            DataClassification = CustomerContent;
        }
        field(4; Power; Integer)
        {
            Caption = 'Power (CV)';
            DataClassification = CustomerContent;
        }
        field(5; "Engine Type"; Enum EngineType)
        {
            Caption = 'Engine Type';
            DataClassification = CustomerContent;
        }
    }
}
```



```

    keys
    {
        key(PK; ModelNo)
        {
            Clustered = true;
        }
    }
}

```

2. The Engine Type field is of the Enum EngineType type, and the enum is defined as follows:

```

enum 50200 EngineType
{
    Extensible = true;
    value(0; Petrol)
    {
        Caption = 'Petrol';
    }
    value(1; Diesel)
    {
        Caption = 'Diesel';
    }
    value(2; Electric)
    {
        Caption = 'Electric';
    }
    value(3; Hybrid)
    {
        Caption = 'Hybrid';
    }
}

```

3. We also create a Car List page (a standard list page) for managing Car data in Dynamics 365 Business Central.
4. Now, we need to create the API page (by using the tpage snippet and then selecting **Page of type API**). The CarAPI page is defined as follows:

```

page 50200 CarAPI
{
    PageType = API;
    Caption = 'Car API';
    APIPublisher = 'sd';
}

```

```

APIGroup = 'custom';
APIVersion = 'v1.0';
EntityName = 'car';
EntitySetName = 'cars';
SourceTable = Car;
DelayedInsert = true;
ODataKeyFields = systemId;

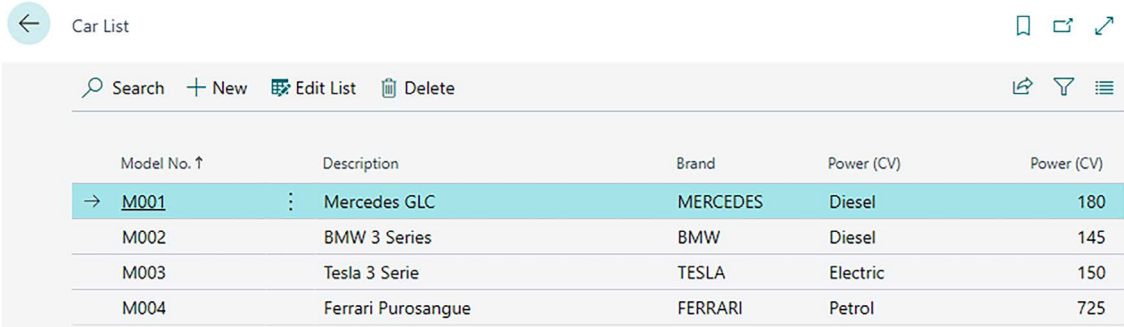
layout
{
    area(Content)
    {
        repeater(GroupName)
        {
            field(modelno; ModelNo)
            {
                Caption = 'modelNo', Locked = true;
            }
            field(description; Description)
            {
                Caption = 'description', Locked = true;
            }
            field(brand; Brand)
            {
                Caption = 'brand', Locked = true;
            }
            field(engineType; "Engine Type")
            {
                Caption = 'engineType', Locked = true;
            }
            field(power; Power)
            {
                Caption = 'power', Locked = true;
            }
            field(systemId; Rec.SystemId)
            {
                Caption = 'systemId', Locked = true;
            }
        }
    }
}

```

This page exposes the fields that we want to have in our API by applying the naming rules according to the OData specifications. Here, you can see that we have defined some important parameters:

- **APIPublisher:** Sets the publisher of the API endpoint that the page is exposed in
- **APIGroup:** Sets the group of the API endpoint that the page is exposed in
- **APIVersion:** Sets the version(s) of the API endpoint the page is exposed in
- **EntityName:** Sets the singular entity name with which the page is exposed in the API endpoint
- **EntitySetName:** Sets the plural entity name with which the page is exposed in the API endpoint
- **SourceTable:** Sets the table name from which this page will display records
- **ODataKeyFields:** Specifies the fields to select when using OData

Now, press F5 in Visual Studio Code and publish your extension. When it's published, search for Car List and then insert some example Car records, such as the following:



The screenshot shows a web application titled 'Car List'. It features a search bar, a '+ New' button, and buttons for 'Edit List' and 'Delete'. Below the navigation bar is a table with five columns: 'Model No. ↑', 'Description', 'Brand', 'Power (CV)', and 'Power (CV)'. The table contains four rows of data. The first row is highlighted in light blue and includes a right-pointing arrow in the 'Model No.' column.

Model No. ↑	Description	Brand	Power (CV)	Power (CV)
→ M001	Mercedes GLC	MERCEDES	Diesel	180
M002	BMW 3 Series	BMW	Diesel	145
M003	Tesla 3 Serie	TESLA	Electric	150
M004	Ferrari Purosangue	FERRARI	Petrol	725

Figure 13.21: Inserting Car records

Now, we can test our custom API. When published in a SaaS tenant, a custom API endpoint has the following format:

```
{BaseURL}/api/<api publisher>/<api group>/<api version>
```

Here, {BaseURL} is `https://api.businesscentral.dynamics.com/v2.0/ENVIRONMENTNAME`.

If you're testing it on a Docker-based sandbox or in an on-premises environment, the API endpoint is like this:

```
{ServerUrl:ODATA_Port}/{ServerInstance}/api//<api publisher>/<api group>/<api version>
```

The APIs are invoked per company, as previously explained, and you need to first retrieve the company ID of the desired company.

To have the list of cars for the selected company, you need to send a GET request to the following URL (by passing the company ID that was retrieved with the preceding call):

```
{baseUrl}/api/sd/custom/v1.0/companies({id})/cars
```

This is the response that we get:

```

{
  "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/USSandbox/api/sd/custom/v1.0/$metadata#comp
anies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/cars",
  "value": [
    {
      "@odata.etag": "W/\"JzE50zg10TQ5MTY1ODQ0OTg5NjYxNzQxOzAwOyc=\"",
      "systemId": "24e5343d-a927-ee11-bdf5-00224828e836",
      "modelName": "M001",
      "description": "Mercedes GLC",
      "brand": "MERCEDES",
      "engineType": "Diesel",
      "power": 180
    },
    {
      "@odata.etag": "W/\"JzE50zgZMzQ2ODgzNDQzOTM0MzUzMTYxOzAwOyc=\"",
      "systemId": "f91e104d-a927-ee11-bdf5-00224828e836",
      "modelName": "M002",
      "description": "BMW 3 Serie",
      "brand": "BMW",
      "engineType": "Diesel",
      "power": 145
    },
    {
      "@odata.etag": "W/\"JzE50zMyNjg4Mzc4NjYzNzUyMjQyNTEwOzAwOyc=\"",
      "systemId": "0528e058-a927-ee11-bdf5-00224828e836",
      "modelName": "M003",
      "description": "Tesla 3 Serie",
      "brand": "TESLA",
      "engineType": "Electric",
      "power": 150
    },
    {
      "@odata.etag": "W/\"JzE40zM5NjAyNDQ4MjAyNDQxNTQxNTE7MDA7Jw==\"",
      "systemId": "e1df7062-a927-ee11-bdf5-00224828e836",
      "modelName": "M004",
      "description": "Ferrari Purosangue",
      "brand": "FERRARI",
      "engineType": "Petrol",
      "power": 725
    }
  ]
}

```

Figure 13.22: Retrieving Car records with a GET request

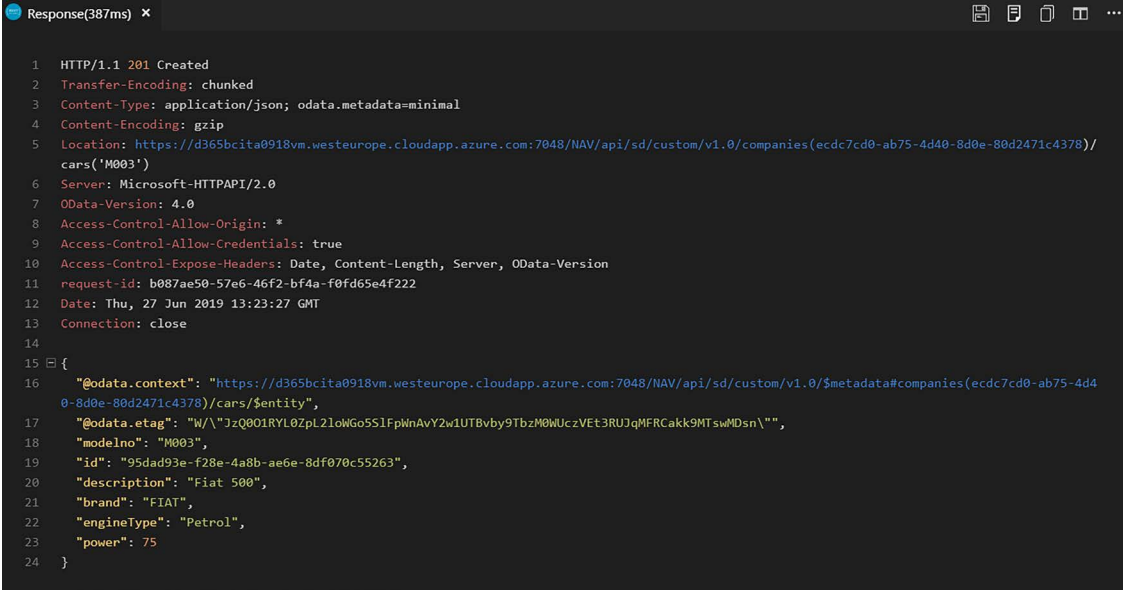
As you can see, we have the JSON representation of the inserted records, and every field (JSON token) has the name that we assigned in our API definition.

To insert a new Car record via our previously published custom cars API, you need to send a POST request to the following URL by passing the JSON of the Car record to create in the body:

```
POST {{baseurl}}/api/sd/custom/v1.0/companies({{companyid}})/cars
Content-Type: application/json
Authorization: Bearer {{token}}

{
  "modelName": "M005",
  "description": "Fiat 500",
  "brand": "FIAT",
  "engineType": "Petrol",
  "power": 75
}
```

The response received is as follows:



```
Response(387ms) X
1 HTTP/1.1 201 Created
2 Transfer-Encoding: chunked
3 Content-Type: application/json; odata.metadata=minimal
4 Content-Encoding: gzip
5 Location: https://d365bcita0918vm.westeurope.cloudapp.azure.com:7048/NAV/api/sd/custom/v1.0/companies(ecdc7cd0-ab75-4d40-8d0e-80d2471c4378)/cars('M003')
6 Server: Microsoft-HTTPAPI/2.0
7 OData-Version: 4.0
8 Access-Control-Allow-Origin: *
9 Access-Control-Allow-Credentials: true
10 Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version
11 request-id: b087ae50-57e6-46f2-bf4a-f0fd65e4f222
12 Date: Thu, 27 Jun 2019 13:23:27 GMT
13 Connection: close
14
15 {
16   "@odata.context": "https://d365bcita0918vm.westeurope.cloudapp.azure.com:7048/NAV/api/sd/custom/v1.0/$metadata#companies(ecdc7cd0-ab75-4d40-8d0e-80d2471c4378)/cars/$entity",
17   "@odata.etag": "W/\"JzQ001RYL0ZpL2loWGo5S1FpWnAvY2w1UTBvby9TbzM0WUc2VET3RUJqMFRkCakk9MTswMDsn\"",
18   "modelName": "M003",
19   "id": "95dad93e-f28e-4a8b-ae6e-8df070c55263",
20   "description": "Fiat 500",
21   "brand": "FIAT",
22   "engineType": "Petrol",
23   "power": 75
24 }
```

Figure 13.23: Inserting Car records with a POST request

We receive HTTP/1.1 201 Created and the JSON details of the Car record are added to Dynamics 365 Business Central.

If you look at Car List in Dynamics 365 Business Central, you can see that a new record has been created:

← CAR LIST | WORK DATE: 4/1/2019

Search

+ New

Edit List

Delete

Open in Excel

Filter

Menu

MODEL NO.	DESCRIPTION	BRAND	POWER (CV)	POWER (CV)
M001	Mercedes GLC	MERCEDES	Diesel	180
M002	BMW 3 Series	BMW	Diesel	145
M003	Fiat 500	FIAT	Petrol	75

Figure 13.24: Viewing the new record



When sending a POST request, remember to correctly set the content type of the request to `application/json`. Otherwise, you can receive a quite confusing error in the response message, such as `{"error":{"code":"BadRequest","message":"Cannot create an instance of an interface."}}`.

To retrieve the details of a specific car record, just send a GET request to the following URL:

```
{baseUrl}/api/sd/custom/v1.0/companies({id})/cars({id})
```

This is done by passing the GUID of the car record to retrieve.

In our example, if we want to retrieve the details of the Mercedes record, we have to send an HTTP GET request to the following URL:

```
{BaseUrl}/api/sd/custom/v1.0/companies(CompanyID)/cars(24e5343d-a927-ee11-bdf5-00224828e836)
```

Here, 24e5343d-a927-ee11-bdf5-00224828e836 is the `SystemId` of the record we want to retrieve. This is the response we receive:

```

HTTP/1.1 200 OK
Transfer-Encoding: chunked
Content-Type: application/json; odata.metadata=minimal; odata.streaming=true
Content-Encoding: gzip
Vary: Accept-Encoding
Server: Microsoft-HTTPAPI/2.0
Access-Control-Allow-Headers: Origin, X-Requested-With, Authorization
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
ms-correlation-x: 00ad5e39-775e-5c58-dd15-7f4e3019a41d
mise-correlation-id: 4aeb62f1-02fd-47b4-9c0d-3816928891ab
OData-Version: 4.0
Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version, ms-correlation-x
request-id: 7892e2de-cba2-468f-bd75-5c79003d7883
x-content-type-options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubDomains
Date: Wed, 26 Jul 2023 09:07:11 GMT
Connection: close

{
  "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/US Sandbox/api/sd/custom/v1.0/$metadata#companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/cars/$entity",
  "@odata.etag": "W/\"JzE50zg10TQ5MTY1ODQ0OTg5NjYxNzQxOzAwOyc=\"",
  "systemId": "24e5343d-a927-ee11-bdf5-00224828e836",
  "modelName": "M001",
  "description": "Mercedes GLC",
  "brand": "MERCEDES",
  "engineType": "Diesel",
  "power": 180
}

```

Figure 13.25: Retrieving Car record details with a GET request

As you can see, we have retrieved the JSON representation of the specified Car record.

Implementing a new API for an existing entity

As we discussed in the *Using the OData protocol for APIs* section, you cannot extend an existing standard Dynamics 365 Business Central API page. If you need to retrieve new fields for a standard Dynamics 365 Business Central entity, you need to create a new API page in your namespace.

For example, here, I am creating a simple new API that retrieves a customer's details, which are not natively exposed in the standard Customer API. The API page is defined as follows:

```

page 50115 PKTCustomerAPI
{
    PageType = API;
    Caption = 'customer';
    APIPublisher = 'PACKT';
}

```

```
APIVersion = 'v1.0';
APIGroup = 'customapi';
EntityName = 'customer';
EntitySetName = 'customers';
SourceTable = Customer;
DelayedInsert = true;
ODataKeyFields = SystemId;

layout
{
    area(Content)
    {
        repeater(GroupName)
        {
            field(no; Rec."No.")
            {
                Caption = 'no', Locked = true;
            }
            field(name; Rec.Name)
            {
                Caption = 'name', Locked = true;
            }
            field(Id; Rec.SystemId)
            {
                Caption = 'Id', Locked = true;
            }
            field(balanceDue; Rec."Balance Due")
            {
                Caption = 'balanceDue', Locked = true;
            }
            field(creditLimit; Rec."Credit Limit (LCY)")
            {
                Caption = 'creditLimit', Locked = true;
            }
            field(currencyCode; Rec."Currency Code")
            {
                Caption = 'currencyCode', Locked = true;
            }
            field(email; Rec."E-Mail")
            {
                Caption = 'email', Locked = true;
            }
        }
    }
}
```



```

    }
    field(balance; Rec."Balance (LCY)")
    {
        Caption = 'balance', Locked = true;
    }
    field(countryRegionCode; Rec."Country/Region Code")
    {
        Caption = 'countryRegionCode', Locked = true;
    }
    field(netChange; Rec."Net Change")
    {
        Caption = 'netChange', Locked = true;
    }
    field(noOfOrders; Rec."No. of Orders")
    {
        Caption = 'noOfOrders', Locked = true;
    }
    field(noOfReturnOrders; Rec."No. of Return Orders")
    {
        Caption = 'noOfReturnOrders', Locked = true;
    }
    field(phoneNo; Rec."Phone No.")
    {
        Caption = 'phoneNo', Locked = true;
    }
    field(salesLCY; Rec."Sales (LCY)")
    {
        Caption = 'salesLCY', Locked = true;
    }
    field(shippedNotInvoiced; Rec."Shipped Not Invoiced")
    {
        Caption = 'shippedNotInvoiced', Locked = true;
    }
}
}
}
}

```

When this is published on our Dynamics 365 Business Central tenant, we can reach this API at the following endpoint:

```
{baseUrl}/api/PACKT/customapi/v1.0/companies({id})/customers
```

If we send an HTTP GET request to this endpoint to retrieve the Customer records, we get the following API response:

```

{
  "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/USSandbox/api/packt/customapi/v1.0/$metadata#companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/customers",
  "value": [
    {
      "@odata.etag": "W/\"JzIwOzEwMDIzOTQ3MDC4OTQzNTgyMjk5MTswMDsn\"",
      "Id": "71b117ef-e06c-ed11-81b5-6045bd8e591f",
      "no": "10000",
      "name": "Adatum Corporation",
      "balanceDue": 0,
      "creditLimit": 0,
      "currencyCode": "",
      "email": "robert.townes@contoso.com",
      "balance": 0,
      "countryRegionCode": "US",
      "netChange": 0,
      "noOfOrders": 2,
      "noOfReturnOrders": 0,
      "phoneNo": "",
      "salesLCY": 223598.4,
      "shippedNotInvoiced": 0
    },
    {
      "@odata.etag": "W/\"JzE5OzE0NjM0MDYwMTIzMDUyMTgwNzcxOzAwOyc=\"\"",
      "Id": "72b117ef-e06c-ed11-81b5-6045bd8e591f",
      "no": "20000",
      "name": "Trey Research",
      "balanceDue": 3036.6,
      "creditLimit": 0,
      "currencyCode": "",
      "email": "helen.ray@contoso.com",
      "balance": 3036.6,
      "countryRegionCode": "US",
      "netChange": 3036.6,
      "noOfOrders": 0,
      "noOfReturnOrders": 0,
      "phoneNo": "",
      "salesLCY": 58673,
      "shippedNotInvoiced": 0
    }
  ]
}

```

Figure 13.26: Retrieving customer records with a GET request using the custom API

As you can see, the custom API shows all of the Customer fields we have added to our API page (the Normal and FlowFields fields).

A standard API page (like the one just created) supports inserting, modifying, and deleting a record via corresponding HTTP actions, and often you want to control that.

You can control the HTTP actions that the API supports (GET, POST, PATCH, DELETE, etc.) by using the `InsertAllowed`, `ModifyAllowed`, and `DeleteAllowed` properties of the API page.

Using the read-only database replica

Dynamics 365 Business Central SaaS has a read-only replica of the production database active by default. When you have APIs that only need to read data, it's recommended to create APIs that work on this replica and not directly in the production database. This helps you maximize performance and avoid having high loads on the production database (where the users daily work).

To create an API that works on the read-only replica of the database (and so only supports reading data), you need to use the `DataAccessIntent` property of the API page as follows:

```
page 50202 PKTCustomerAPI
{
    PageType = API;
    Caption = 'customer';
    APIPublisher = 'PACKT';
    APIVersion = 'v1.0';
    APIGroup = 'customapi';
    EntityName = 'customer';
    EntitySetName = 'customers';
    SourceTable = Customer;
    ODataKeyFields = SystemId;
    DataAccessIntent = ReadOnly;
    Editable = false;
```

The `DataAccessIntent` property sets the data access intent of the page, and it can have the following values:

- `ReadWrite`: This permits accessing and modifying records (works on the production database).
- `ReadOnly`: This permits accessing records but not modifying them (works on the read-only database replica).

Dynamics 365 APIs' operational limits

Remember that Dynamics 365 Business Central limits the number of simultaneous API calls that you can perform in a certain sliding window. If you have an external service that performs too many requests on a tenant, you could receive an HTTP 429 error (Too many requests reached):

```
{
  "error":
  {
    "code": "Application_TooManyRequests",
    "message": "Too many requests reached. Actual (101). Maximum (100)."
```

The purpose of this is mainly to avoid things such as **Denial-of-Service (DoS)** attacks and insufficient resources in the tenant.

The actual permitted maximum number of requests per minute is as follows:

- **Web service requests: 6,000** web service requests per **user** in the previous **5-minute sliding window**.
- **Concurrency limit for web service requests: 100** concurrently handled (5 processed, 95 queued) web service requests per **user**.

You can see some numbers about Dynamics 365 Business Central at the following link: <https://demiliani.com/2023/11/07/dynamics-365-business-central-is-my-on-premises-customer-ready-for-saas/>.

To avoid this situation, you should handle how you perform requests to a Dynamics 365 Business Central API endpoint and, if you receive this error, you should adopt something such as a **retry policy** on the API calls in your external application.

Using OData bound actions

We can use Dynamics 365 Business Central APIs not only to perform CRUD operations on an entity but also to invoke standard business logic defined in the application (both custom and standard code).

We can trigger Dynamics 365 Business Central business logic associated with the current entity via APIs by using OData **bound actions**. Bound actions are essentially actions that you can invoke via HTTP calls for a selected entity.

Imagine that you want to have an API action to clone a selected customer. This action takes the context of the selected customer and then it creates a new customer record with the same field values (except the Name field).

To do that, we can add a bound action to the previously created Customer API.

To use OData V4 bound actions, you need to declare a procedure in the API page decorated with the [ServiceEnabled] attribute. The procedure is defined as follows:

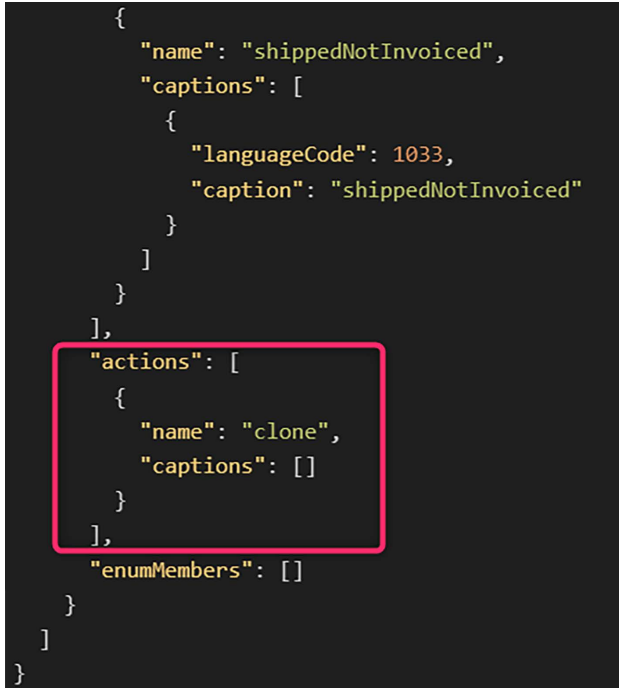
```
[ServiceEnabled]
procedure Clone (var ActionContext: WebServiceActionContext)
var
    Customer: Record Customer;
begin
    Customer.Init();
    Customer.TransferFields(Rec, false);
    Customer.Name := 'CUSTOMER CLONED VIA BOUND ACTION';
```

```
Customer.Insert(true);
    ActionContext.SetResultCode(WebServiceActionResultCode::Created);
end;
```

We can inspect the OData entity definition by sending a GET request to the following endpoint:

```
GET {{baseurl}}/api/packt/customapi/v1.0/entityDefinitions
```

From the response, you can see that now our Customer API exposes an action. Please note that the action name starts with a lowercase letter (clone) while our AL method has a capital letter (Clone):



```
{
  "name": "shippedNotInvoiced",
  "captions": [
    {
      "languageCode": 1033,
      "caption": "shippedNotInvoiced"
    }
  ]
},
  "actions": [
    {
      "name": "clone",
      "captions": []
    }
  ],
  "enumMembers": []
}
```

Figure 13.27: “clone” action exposed by the custom API

To call our bound action, we need to send a POST request to the following endpoint:

```
POST {{baseurl}}/api/packt/customapi/v1.0/companies({{companyid}})/
customers(71b117ef-e06c-ed11-81b5-6045bd8e591f)/Microsoft.NAV.clone
```

Here we’re calling the clone action by passing the SystemId of the customer record to clone (which, in this case, is **Adatum Corporation**).

The AL procedure is invoked and a Customer record has been created (that is, cloned by the customer with "No." = 10000 as the input):

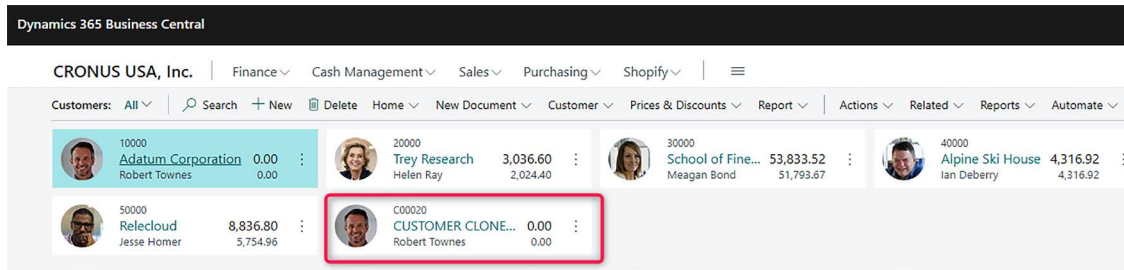


Figure 13.28: Testing the clone action

Here, you learned how to invoke actions linked to a particular record via an API.

In the next section, we'll see how to invoke custom AL code (business process not linked to an entity) via OData APIs.

Using OData unbound actions

OData **unbound actions** are actions that you can invoke via HTTP requests and that are not bound to a particular entity. Unbound actions are useful to execute custom Dynamics 365 Business Central code from external applications.

To use **unbound actions** in Dynamics 365 Business Central, you need to:

1. Create a codeunit exposing your custom actions (procedures).
2. Register and publish the codeunit as a web service from Dynamics 365 Business Central.
3. Invoke the action via a POST OData call (passing the required parameters in the request body).

Imagine that we have an AL procedure that calculates the total amount of sales orders for a particular customer. The procedure is defined in a codeunit object as follows:

```
codeunit 50200 "PKT Unbound Actions"
{
    procedure GetSalesAmountForDay(CustomerNo: Code[20]) totalSales: Decimal;
    var
        SalesLine: Record "Sales Line";
    begin
        SalesLine.SetRange("Document Type", SalesLine."Document Type"::Order);
        SalesLine.SetRange("Sell-to Customer No.", CustomerNo);
        if SalesLine.FindSet() then
            repeat
                totalSales += SalesLine."Line Amount";
            until SalesLine.Next() = 0;
    end
}
```

```
end;
}
```

When the extension containing that codeunit is published, we can open the **Web Services** page in Dynamics 365 Business Central and publish the codeunit as an OData endpoint as follows:

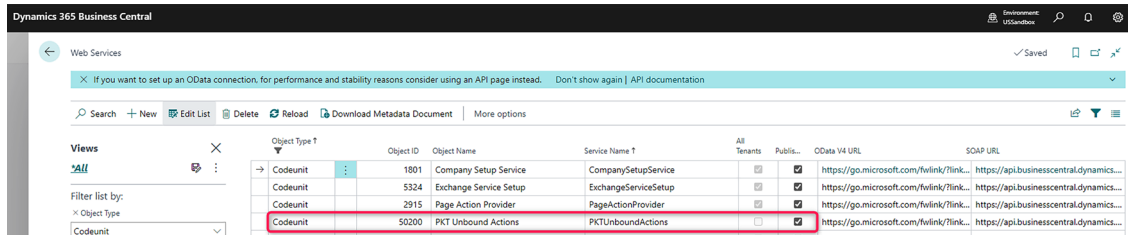


Figure 13.29: Publishing a codeunit as an OData endpoint

Here, you need to:

1. Set **Object Type** as **Codeunit**.
2. Set **Object ID** as the ID of the codeunit object that you want to expose.
3. Set **Service Name** as the name of the exposed service (don't use spaces). This will be part of your OData endpoint.
4. Set **Published** as **true**.

To call the published codeunit method as an OData unbound action, you need to send a POST request to the following endpoint:

```
POST {{baseurl}}/ODataV4/PKTUnboundActions_GetSalesAmountForCustomer?company=18bb64bd-e06c-ed11-81b5-6045bd8e591f
```

Unbound actions must have the company ID as a parameter in the request URL. As previously said, if the procedure that you want to call requires parameters, you need to pass them in the JSON body of the request. Please note that the parameter names must be passed with a lowercase starting letter.

This is the complete call of our previously published action:

```
POST {{baseurl}}/ODataV4/PKTUnboundActions_GetSalesAmountForCustomer?company=18bb64bd-e06c-ed11-81b5-6045bd8e591f
Authorization: Bearer {{token}}
Content-Type: application/json

{
  "customerNo": "10000"
}
```

This is the response, showing the total sales order amount for the selected customer:

```
HTTP/1.1 200 OK
Transfer-Encoding: chunked
Content-Type: application/json; odata.metadata=minimal; odata.streaming=true
Content-Encoding: gzip
Vary: Accept-Encoding
Server: Microsoft-HTTPAPI/2.0
Access-Control-Allow-Headers: Origin, X-Requested-With, Authorization
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true
ms-correlation-x: e84a5f3b-c70b-0555-4a84-57920b3d4f80
mise-correlation-id: 53e953a6-8c4c-440e-8e80-d7b98c8e732a
OData-Version: 4.0
Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version, ms-correlation-x
request-id: 9bda8231-3569-40d4-864e-2f8d8ee49c7c
x-content-type-options: nosniff
Strict-Transport-Security: max-age=31536000; includeSubDomains
Date: Thu, 27 Jul 2023 10:08:01 GMT
Connection: close

~{
  "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/US Sandbox/odata/v4/$metadata#Edm.
  Decimal",
  "value": 19052.9
}
```

Figure 13.30: Response when calling a codeunit with a POST request

By using **unbound actions**, you have the power to call every code you want from external applications, opening a wide range of integration scenarios.

In the next section, we'll see how to send requests in batches to an API endpoint.

Using OData batch calls with Dynamics 365 Business Central APIs

Usually, when calling Dynamics 365 Business Central APIs, you perform a single operation at a time (you send an API request and you receive a response, then you send another API request, and so on).

OData also supports batch requests. With batch requests, you can combine multiple operations in a single OData request. The batch request is sent as a single POST operation to the OData \$batch endpoint, and the body of this request must contain a JSON with the set of operations to perform. This permits you to minimize the number of HTTP calls (requests) that you send to an API endpoint.

All APIs (standard and custom) in Dynamics 365 Business Central support batch requests. The \$batch endpoint is available at the following URLs:

- Standard APIs: `https://{baseurl}/api/v2.0/$batch`
- Custom APIs: `https://{baseurl}/api/[publisher]/[apigroup]/[version]/$batch`

When calling the `$batch` endpoint, you should always specify that you want a response in JSON format, so set the `Accept` header parameter as `application/json`:

```
POST {{baseurl}}/api/v2.0/$batch
Content-Type: application/json
Accept: application/json
```

When using batch requests, the request body must be a JSON object containing a `requests` array parameter of operations:

```
{
  "requests": []
}
```

Each request must have the following properties:

- **id** (mandatory): The unique identifier for the operation.
- **method** (mandatory): The HTTP method of the operation (GET, POST, PATCH, PUT, or DELETE).
- **url** (mandatory): The URL of the API to call (contains parameters like `$select`, `$filter`, etc.). This can be a relative path or an absolute path:
 - A relative path replaces the `$batch` part of the batch request URL.
 - An absolute path replaces the complete URL of the batch request.
- **headers** (optional): A list of the headers for the request.
- **body** (optional): The JSON body of the specific request.

As an example of how to compose a batch request, let's imagine inserting multiple journal lines via the standard Dynamics 365 Business Central `journalLines` API. We can send a batch request like the following:

```
POST {{baseurl}}/api/v2.0/$batch
Authorization: Bearer {{token}}
Content-Type: application/json
Accept: application/json

{
  "requests": [
    {
      "method": "POST",
      "id": "r1",
      "url": "companies({{companyId}})/journals({{journalId}})/
journalLines",
      "headers": {
        "Content-Type": "application/json"
      },

```

```

        "body": {
            "accountId": "{{accountId}}",
            "postingDate": "2023-07-27",
            "documentNumber": "JNL2023-07-01",
            "amount": -5000,
            "description": "Salary to Consultants"
        }
    },
    {
        "method": "POST",
        "id": "r2",
        "url": "companies({{companyId}})/journals({{journalId}})/
journalLines",
        "headers": {
            "Content-Type": "application/json"
        },
        "body": {
            "accountId": "{{accountId}}",
            "postingDate": "2023-07-27",
            "documentNumber": "JNL2023-07-02",
            "amount": -7500,
            "description": "Salary to Developers"
        }
    }
]
}

```

Here, we're sending two journal lines POST requests (named r1 and r2) with their details in a single batch call.

The API response is as follows:

```
"responses": [
  {
    "id": "r1",
    "status": 201,
    "headers": {
      "urls-rewritten-to-public": "false",
      "location": "https://api.businesscentral.dynamics.com/v2.0/USSandbox/api/v2.0/companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/journals(e4700afb-e06c-ed11-81b5-6045bd8e591f)/journalLines(5479fa3d-7e2c-ee11-bdf5-000d3a16ab42)",
      "content-type": "application/json; odata.metadata=minimal; odata.streaming=true",
      "odata-version": "4.0"
    },
    "body": {
      "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/USSandbox/api/v2.0/$metadata#companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/journals(e4700afb-e06c-ed11-81b5-6045bd8e591f)/journalLines/$entity",
      "@odata.etag": "W/\"JzIwOzE3NDASNzg5MzAzOTQ3NzI1MjVjMTswMDsn\"",
      "id": "5479fa3d-7e2c-ee11-bdf5-000d3a16ab42",
      "journalId": "e4700afb-e06c-ed11-81b5-6045bd8e591f",
      "journalDisplayName": "DEFAULT",
      "lineNumber": 10000,
      "accountType": "G_x002F_L_x0020_Account",
      "accountId": "2ab117ef-e06c-ed11-81b5-6045bd8e591f",
      "accountNumber": "10000",
      "postingDate": "2023-07-27",
      "documentNumber": "JNL2023-07-01",
      "externalDocumentNumber": "",
      "amount": -5000.00,
      "description": "Salary to Consultants",
      "comment": "",
      "taxCode": "",
      "balanceAccountType": "G_x002F_L_x0020_Account",
      "balancingAccountId": "00000000-0000-0000-0000-000000000000",
      "balancingAccountNumber": "",
      "lastModifiedDateTime": "2023-07-27T13:05:22.307Z"
    }
  },
  {
    "id": "r2",
    "status": 201,
    "headers": {
      "urls-rewritten-to-public": "false",
      "location": "https://api.businesscentral.dynamics.com/v2.0/USSandbox/api/v2.0/companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/journals(e4700afb-e06c-ed11-81b5-6045bd8e591f)/journalLines(5579fa3d-7e2c-ee11-bdf5-000d3a16ab42)",
      "content-type": "application/json; odata.metadata=minimal; odata.streaming=true",
      "odata-version": "4.0"
    },
    "body": {
      "@odata.context": "https://api.businesscentral.dynamics.com/v2.0/USSandbox/api/v2.0/$metadata#companies(18bb64bd-e06c-ed11-81b5-6045bd8e591f)/journals(e4700afb-e06c-ed11-81b5-6045bd8e591f)/journalLines/$entity",
      "@odata.etag": "W/\"JzIwOzE2MTM1ODc4NzU1MDAxNTU2OTI5MTswMDsn\"",
      "id": "5579fa3d-7e2c-ee11-bdf5-000d3a16ab42",
      "journalId": "e4700afb-e06c-ed11-81b5-6045bd8e591f",
      "journalDisplayName": "DEFAULT",
      "lineNumber": 20000,
```

Figure 13.31: API response to a POST request

As you can see, we have a single JSON response that contains an array of responses (a response for each operation we sent) with their status and their body. Each single result has the corresponding request ID identifier. Please remember that the order of the responses can be different from the order of the request you send.

When you send a batch of requests, some of them can be successfully executed and some of them can fail. There are scenarios where you also want to continue processing the other requests in the batch if a previous request fails, and there are other scenarios where you want to stop and completely roll back the entire batch.

If you want to continue to process the requests in the batch if a request fails, you need to set the `Prefer` header as follows:

```
POST {{baseurl}}/api/v2.0/$batch
Content-Type: application/json
Accept: application/json
Prefer: odata.continue-on-error
```

If you want a transactional behavior, you need to use the `OData-Isolation` header, as in the following:

```
POST {{baseurl}}/api/v2.0/$batch
Content-Type: application/json
Accept: application/json
Prefer: odata.continue-on-error
OData-Isolation: snapshot
```

In this way, you're using OData snapshot isolation. By also maintaining the `Prefer: odata.continue-on-error` parameter, you will obtain a full list of all the failing operations (instead of only the first one).

A batch API call can also contain different types of operations. You can, for example, send a single batch API request that contains operations for creating a customer, creating a vendor, creating a sales order for a new customer, and so on.

When sending multiple operations in a batch call, you need to remember that operations are always processed in parallel. But there are scenarios where this is not possible.

As an example, let's consider our previous batch example, where we create two journal lines in a single API request. Now imagine that we want also to post them, and the call to the posting method must be done in the same batch API request.

As you can imagine, we cannot post the journal lines before they are successfully inserted, so the posting operations must be **dependent** on the success of the previous insert requests. To specify that an operation in the batch should not start before the other operations, you can use the `dependsOn` property. The batch API request for this case will be like the following:

```
POST {{baseurl}}/api/v2.0/$batch
Authorization: Bearer {{token}}
Content-Type: application/json
```

Accept: application/json

```
{
  "requests": [
    {
      "method": "POST",
      "id": "r1",
      "url": "companies({{companyId}})/journals({{journalId}})/
journalLines",
      "headers": {
        "Content-Type": "application/json"
      },
      "body": {
        "accountId": "{{accountId}}",
        "postingDate": "2023-07-27",
        "documentNumber": "JNL2023-07-01",
        "amount": -5000,
        "description": "Salary to Consultants"
      }
    },
    {
      "method": "POST",
      "id": "r2",
      "url": "companies({{companyId}})/journals({{journalId}})/
journalLines",
      "headers": {
        "Content-Type": "application/json"
      },
      "body": {
        "accountId": "{{accountId}}",
        "postingDate": "2023-07-27",
        "documentNumber": "JNL2023-07-02",
        "amount": -7500,
        "description": "Salary to Developers"
      }
    },
    {
      "method": "POST",
      "id": "4",
      "dependsOn": ["r1", "r2"],
```

```

        "url": "companies({{companyId}})/journals({{journalId}})/
Microsoft.NAV.post",
        "headers": {
            "Content-Type": "application/json"
        },
        "body": { }
    }
]
}

```

We explored the possibilities for handling batch calls via OData APIs. Batch calls are useful when you want to optimize sending multiple requests to an API endpoint, and in some business scenarios, they can be extremely useful to use.

In the next section, we'll examine the concept of webhooks inside Dynamics 365 Business Central and we'll explore how to subscribe to notifications sent from a Dynamics 365 Business Central entity.

Using Dynamics 365 Business Central webhooks

Webhooks are a way to create event-driven service integrations: instead of polling another system to check whether there are any changes in entities with webhooks, a client subscribes to events that will be pushed to it from the source system. Dynamics 365 Business Central supports webhooks, so a client can subscribe to a webhook notification (event) and will then automatically receive notifications if an entity in Dynamics 365 Business Central changes.

To use webhooks with Dynamics 365 Business Central, we need to perform the following steps:

1. A subscriber must register the webhook subscription with Dynamics 365 Business Central by making a POST request to the **Subscription** API and by passing a notification URL in the request body. The endpoint URL is as follows:

```

https://api.businesscentral.dynamics.com/v2.0/TENANTID/production/api/
v1.0/subscriptions

```

2. The request body to establish a subscription is the following (here, we're using the customers entity as an example):

```

{
    "notificationUrl": "YourApplicationUrl",
    "resource": "https://api.businesscentral.dynamics.com/v2.0/TENANTID/
production/api/v1.0/companies(COMPANYID)/customers",
    "clientState": "SomeSharedSecretForTheNotificationUrl"
}

```

3. Dynamics 365 Business Central returns a validation token to the subscriber.
4. The subscriber needs to return the validation token in the response body and provide the status code 200 (this is the mandatory handshake phase).

5. If Dynamics 365 Business Central receives the validation token in the response body, the subscription is registered and notifications will be sent to the notification URL.

When a subscription is established, the subscriber receives a notification for every update on the subscribed entity. Webhook subscriptions expire after 3 days if they are not renewed before that.

To renew a webhook subscription, a subscriber must send a PATCH request to the subscription endpoint (this request requires the handshake phase, too). The request endpoint to renew a webhook subscription is as follows:

```
https://api.businesscentral.dynamics.com/v2.0/TENANTID/production/api/v1.0/subscriptions('SUBSCRIPTIONID')
```



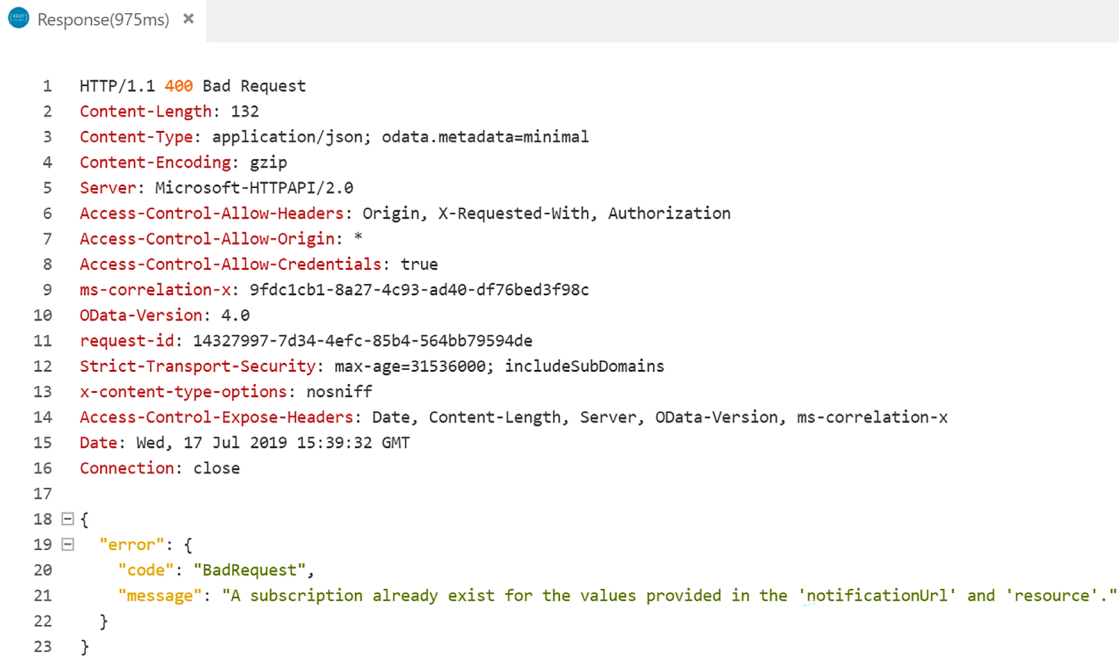
To renew a webhook subscription, you need to pass the `@odata.etag` tag of your previously established subscription as an If-Match block in the PATCH request header.

This is the HTTP response that you receive when the subscription is established:

```
Response(1635ms) x
6  Server: Microsoft-HTTPAPI/2.0
7  Access-Control-Allow-Headers: Origin, X-Requested-With, Authorization
8  Access-Control-Allow-Origin: *
9  Access-Control-Allow-Credentials: true
10 ms-correlation-x: 1a6e52dd-fccc-4692-800f-f7e705b9ed9c
11 OData-Version: 4.0
12 request-id: 28fc1b91-d868-49f4-bc31-c733984438ec
13 Strict-Transport-Security: max-age=31536000; includeSubDomains
14 x-content-type-options: nosniff
15 Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version, ms-correlation-x
16 Date: Wed, 10 Jul 2019 15:45:42 GMT
17 Connection: close
18
19 {
20   "@odata.context": "https://api.businesscentral.dynamics.com/v1.0/194e87bd-73c6-43c6-95d7-1ca48985db5e/api/beta/$metadata#subscriptions/$entity",
21   "@odata.etag": "W/\"JzQ00lyemodod1UxawduR3NZaUp2aW9oVTFERU5vQV1JQXpLQi8rU3NyVzBsNmM9MTswMDsn\"",
22   "subscriptionId": "84765ab3c48c4bb0beeb789f936a8c71",
23   "notificationUrl": "https://d365bcwebhooks.azurewebsites.net/api/D365BCListener?code=MJ4Y2Pltvu0Op/4qz8GbDIQ56LtJqBPiV6Y1fP16cQvXIZL4oA2NRQ==&testerId=subscriber1",
24   "resource": "api/beta/companies(80d28ea6-02a3-4ec3-98f7-936c2000c7b3)/customers",
25   "userId": "10cb2537-0adf-4a69-97e5-894c60788667",
26   "lastModifiedDate": "2019-07-10T15:45:41Z",
27   "clientState": "",
28   "expirationDateTime": "2019-07-13T15:45:41Z"
29 }
```

Figure 13.32: HTTP response to a PATCH request

If you try to issue a subscription request again to the same endpoint where an active subscription has been established, you receive the following error:



```

Response(975ms) ✕
1 HTTP/1.1 400 Bad Request
2 Content-Length: 132
3 Content-Type: application/json; odata.metadata=minimal
4 Content-Encoding: gzip
5 Server: Microsoft-HTTPAPI/2.0
6 Access-Control-Allow-Headers: Origin, X-Requested-With, Authorization
7 Access-Control-Allow-Origin: *
8 Access-Control-Allow-Credentials: true
9 ms-correlation-x: 9fdc1cb1-8a27-4c93-ad40-df76bed3f98c
10 OData-Version: 4.0
11 request-id: 14327997-7d34-4efc-85b4-564bb79594de
12 Strict-Transport-Security: max-age=31536000; includeSubDomains
13 x-content-type-options: nosniff
14 Access-Control-Expose-Headers: Date, Content-Length, Server, OData-Version, ms-correlation-x
15 Date: Wed, 17 Jul 2019 15:39:32 GMT
16 Connection: close
17
18 {
19   "error": {
20     "code": "BadRequest",
21     "message": "A subscription already exist for the values provided in the 'notificationUrl' and 'resource'."
22   }
23 }

```

Figure 13.33: Error message to a PATCH request

When a subscription is established, a subscriber can receive notifications when the subscribed entities in Dynamics 365 Business Central are modified. This is an example of a notification sent to a subscriber (the notification is a JSON object that contains all of the modified entities):

```

{
  "value": [
    {
      "subscriptionId": "customers",
      "clientState": "someClientState",
      "expirationDateTime": "2019-07-20T07:52:31Z",
      "resource": "api/beta/companies(80d28ea6-02a3-4ec3-98f7-936c2000c7b3)/customers(26814998-936a-401c-81c1-0e848a64971d)",
      "changeType": "updated",
      "lastModifiedDateTime": "2019-07-19T12:54:20.467Z"
    }
  ],

```



```
{
  "subscriptionId": "webhookCustomersId",
  "clientState": "someClientState",
  "expirationDateTime": "2019-07-20T07:52:31Z",
  "resource": "api/beta/companies(80d28ea6-02a3-4ec3-98f7-
              936c2000c7b3)/customers(130bbd17-dbb9-4790-9b12-
              2b0e9c9d22c3)",
  "changeType": "created",
  "lastModifiedDateTime": "2019-07-19T12:54:26.057Z"
}
]
```

Webhooks are also exposed for custom objects in our extensions. A page with `PageType = API` will expose webhooks with the following limitations (these also apply to standard API pages):

- The page cannot have composite keys.
- The page cannot use temporary tables or system tables as `SourceTable`.

A subscription to a webhook can be deleted by sending a DELETE request to the `/subscriptions({id})` endpoint. Also, to delete a subscription, you need to send a request with the `If-Match` header containing `@odata.etag` of the subscription to delete.

For more information about Dynamics 365 Business Central webhooks, I recommend checking this post: <http://demiliani.com/2019/12/10/webhooks-with-dynamics-365-business-central/>.

Summary

In this chapter, we gave an overview of how to use the OData stack (and RESTful APIs, in particular) for integration with Dynamics 365 Business Central. We saw how to handle authentication with Microsoft Entra ID, how to use standard APIs, how to create custom APIs, how to create applications that use Dynamics 365 Business Central APIs, and how to use advanced concepts such as batches, bound and unbound actions, and webhooks.

At the end of this chapter, you were given a complete overview of how to expose Dynamics 365 Business Central business logic and entities and how to handle integrations with external applications by using REST HTTP calls and webhooks. APIs are the recommended way for creating integrations between Dynamics 365 Business Central and external applications.

In the next chapter, we'll see how to use Azure Functions and other serverless services with Dynamics 365 Business Central extensions to improve your integrations.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://packt.link/businesscenter>

